

Full length article

Refunded but rewarded: The double dip attack on cashback reward engines

S M Zia Ur Rashid^{*}, Suman Rath

Department of Electrical & Computer Engineering, The University of Tulsa, Tulsa, OK, USA

ARTICLE INFO

Keywords:

Cashback reward abuse
Business logic vulnerability
Double-dip attack
Refund fraud
Cashback reward engines
Payment security

ABSTRACT

Cashback and loyalty reward programs now serve as central instruments in the competitive landscape of cards, digital wallets, and payment platforms. Despite their financial significance, the business logic governing these programs is seldom treated as a security critical surface. In this paper, we study a class of *reward abuse attacks* that arise from flaws in how reward systems accrue, redeem, and adjust incentives when underlying transactions are reversed through refunds. Using controlled, small scale experiments on six issuer accounts we legitimately hold, we document a spectrum of real world behaviors in production systems. At one extreme, a debit based cashback program (Issuer A) never adjusts rewards when refunded transactions post, enabling a deterministic *double dip cashback reward abuse attack*. A credit card program (Issuer B) exhibits an analogous reward integrity violation because rewards are finalized and become redeemable at statement close, whereas refunds arriving after that boundary receive no adjustment. At an intermediate tier, a credit card issuer (Issuer F) creates negative reward entries on refunds at statement close but makes rewards redeemable immediately upon settlement, creating a timing asymmetry that allows users to extract reward value before clawback occurs. At the robust end, three credit card issuers (C, D, and E) implement indefinite negative balance enforcement with proportional clawback, and were robust against the workflows we tested. We formalize reward engines as state machines, introduce two integrity invariants (*Reward Integrity* and *Refund Reward Consistency*), develop a taxonomy of vulnerability classes mapped to CWE and OWASP categories, and present defensive pseudo algorithms with a semi formal correctness argument that holds, under stated assumptions, for transaction separable reward programs. The primary vulnerability (Issuer A) was reported through a private bug bounty program and has been acknowledged by the vendor; good faith disclosure efforts for Issuer B are detailed in Section Section 8.

1. Introduction

Cashback and loyalty features are common across credit card, debit card, and digital wallet products in the United States. These programs collectively move billions of dollars in incentive value each year, and for many issuers they serve as a primary lever to acquire customers, steer spending, and generate interchange revenue (Taleizadeh et al., 2023; Wu et al., 2023). Yet while the payment authorization and settlement systems that underlie these programs are engineered with decades of security scrutiny, the reward engines that sit on top of them receive far less attention. In many organizations, the logic that decides when to grant, adjust, or revoke cashback lives closer to marketing teams than to security engineering.

This paper demonstrates that reward engines can harbor exploitable business logic vulnerabilities with direct financial consequences. We discovered and confirmed, on accounts we personally hold, that certain production cashback programs allow a user to earn rewards on a purchase, redeem those rewards for cash or statement credit, and then

return the merchandise for a full refund without any corresponding adjustment to the reward ledger. The user ends the cycle with zero net spend and positive net reward, a repeatable reward retention outcome that can recur every billing period up to the program's monthly cap.

We use the term *reward abuse attack* to describe systematic, intentional exploitation of cashback and loyalty reward mechanisms to extract monetary benefit beyond the platform's intended design, while using legitimate accounts and nominally valid transactions. This definition distinguishes our concern from classic payment fraud involving stolen or cloned cards, from casual deal hunting that stays within published terms, and from friendly fraud or chargeback fraud where users dispute legitimate charges as unauthorized. Our focus is on logic flaws in how rewards are accounted for in response to refunds, not on identity theft or claim falsification.

Our work is grounded in a concrete pattern we call the *double dip cashback reward abuse attack* (DDRA): a user earns and redeems cashback on a purchase, then later reverses the underlying spend via a

^{*} Corresponding author.

E-mail addresses: ziaur-rashid@utulsa.edu (S M Z U Rashid), suman-rath@utulsa.edu (S. Rath).

<https://doi.org/10.1016/j.cose.2026.105129>

Received 5 April 2026; Received in revised form 22 July 2026; Accepted 20 August 2026

Available online 29 August 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

refund without any proportional clawback of rewards. We discovered two distinct variants of this pattern operating in production financial systems, each exploiting different architectural choices in the reward engine.

We make the following four main contributions:

1. **Empirical case studies.** We instantiate a three tier vulnerability taxonomy using six production issuer accounts we legitimately hold:

- **Case I (Issuer A):** A debit based cashback program that completely fails to adjust rewards when underlying transactions are refunded, enabling a deterministic DDRA.
- **Case II (Issuer B):** A credit card program whose statement cycle reward computation adjusted a refund arriving before statement close but, in the transactions we tested, applied no adjustment to refunds arriving after it, producing an analogous reward integrity violation.
- **Case III (Issuers C, D, E, and F):** Credit card programs exhibiting a range of negative balance designs, from indefinite enforcement observed at Issuers C, D, and E to a partially robust design where instant reward availability combined with batched clawback creates a timing asymmetry (Issuer F).

Our empirical scope reflects the accounts we personally hold and were not selected through systematic sampling. The six issuers span both debit and credit card products from major U.S. financial institutions. The sample is constrained by account creation requirements and is not intended to estimate population prevalence; sample size remains a limitation, which we discuss in Section 8.

2. **Formal model and threat framing.** We model reward engines as state machines over transactions and reward ledgers and define an adversary model that captures an honest but opportunistic consumer who uses only documented features (purchases, refunds, redemption) but strategically chooses timing.
3. **Integrity invariants and vulnerability taxonomy.** We introduce two key security properties, *Reward Integrity* and *Refund Reward Consistency*, and identify several classes of reward logic flaws. We map these to CWE-841 (MITRE Corporation, 2026) and the recently published OWASP Business Logic Abuse Top 10 (OWASP Foundation, 2026).
4. **Defensive algorithms.** We propose implementation oriented pseudo algorithms with a semi formal correctness argument under stated assumptions. The algorithms link rewards to specific transactions, apply proportional clawback on full and partial refunds, support negative reward balances with automatic future offset, and reconcile statement cycle rewards against pending refunds.

Our primary goal is not to expose or criticize specific providers but to elevate cashback reward logic to a first class security topic. We anonymize all providers and omit implementation details that would make identification trivial. We argue that reward engines should be engineered with explicit invariants and state transitions, much like payment authorization logic, and that security reviews should treat reward abuse attacks as seriously as more traditional attack classes.

The remainder of this paper is organized as follows. Section 2 provides background on reward ecosystems, returns abuse, and business logic vulnerabilities. Section 3 presents our system and threat model. Section 4 formalizes integrity invariants and introduces the vulnerability taxonomy. Section 5 details our three case studies. Section 6 presents the formal model and defensive algorithms. Section 7 discusses practical considerations and economic impact. Section 8 covers ethics, disclosure, and methodology. Section 9 surveys related work. Section 10 concludes.

2. Background

Modern payment ecosystems combine card networks, digital wallets, and merchant offers into layered reward structures that operate on top of core payment and refund rails. Consumers routinely earn cashback or loyalty points from their card issuer, and these reward artifacts behave like a soft currency whose lifecycle must track the economic lifecycle of the underlying transaction through authorization, settlement, partial or full refunds, chargebacks, and dispute resolution.

2.1. Card rewards and cashback ecosystems

Card issuers and digital wallets typically implement reward engines that compute benefits as a function of transaction attributes such as merchant category, channel, and spend volume. Common structures include flat rate cashback (for instance, 1 to 2 percent on all purchases), rotating high reward categories (for instance, 5 percent on selected merchants up to a monthly cap), and travel points. These engines may treat refunds and chargebacks as separate transaction events, and must decide when to make rewards available for redemption and how to adjust rewards when the underlying spend is reversed.

The marketing and operations literature treats cashback primarily as a pricing and promotion instrument. Taleizadeh et al. analyze how cashback strategies affect sales under refund policies and customer credit, modeling the joint decisions of price, refund terms, and cashback intensity (Taleizadeh et al., 2023). In subsequent work, the same group study retail pricing, cashback, and refund decisions in a supply chain with both online and direct channels, highlighting the strategic interactions between channel structure, cashback intensity, and refund policies (Taleizadeh et al., 2024). Wu et al. examine a “praise cashback” strategy, where cashback is tied to positive online reviews, and quantify its implications for both consumers and online businesses (Wu et al., 2023). These papers model cashback as a rational incentive mechanism but generally assume compliant consumers and do not treat the reward engine as a security critical state machine that could be abused.

Beyond the pricing and promotion role these models capture, reward programs also serve customer acquisition, the formation of card usage habits, cross selling of other products, and sustained engagement (Vieira et al., 2022). Section 7 returns to the relationship between these objectives and the integrity properties we define.

2.2. Product returns and refund abuse

Product returns and refund abuse have become a major operational concern. Product returns in the United States alone have been estimated in the hundreds of billions of dollars annually, with return rates for online fashion often exceeding 30 to 40 percent (Frei et al., 2020). Harris provides one of the earliest focused studies of fraudulent consumer returns, documenting how consumers exploit liberal return policies and identifying psychological and situational factors that increase such behavior (Harris, 2010). Akturk et al. explicitly model return abuse, both opportunistic and fraudulent, and study how retailers can use technology enabled countermeasures such as customer profiling and product tracking to mitigate abuse (Akturk et al., 2021). Ketzenberg et al. use transaction level data to characterize customer return behaviors, segmenting customers by return propensity (Ketzenberg et al., 2020).

More recent work incorporates return policy abuse as a strategic behavior. Liu and Du focus on e commerce platforms’ return policies under consumer abuse (Liu and Du, 2023). Zhang et al. build a framework of fraudulent returns and mitigation strategies in multichannel retailing (Zhang et al., 2023). Merlano et al. study how consumers perceive anti fraud measures in omnichannel fashion retail (Merlano et al., 2024). Frei et al. explore how the COVID 19 pandemic affected returns management practices (Frei et al., 2023). Collectively, this literature shows that return abuse is economically significant and that

consumers can behave strategically. However, most of these works treat reward programs, if present at all, as exogenous context rather than as a detailed accounting mechanism whose state transitions interact with returns.

2.3. Double dip rewards and cashback abuse

The combination of cashback incentives and liberal return policies creates natural opportunities for financial arbitrage. A user can earn a benefit at the moment of purchase, then later unwind the underlying economic exposure via a refund while attempting to retain the reward. If the reward engine does not treat refunds as first class events, this leads to a double dip pattern: the user briefly commits to a transaction, earns a reward, then exits the transaction while keeping the reward.

Promotion focused research shows that cashback and other incentives can themselves become targets of abuse. Vieira et al. investigate how cashback strategies generate financial benefits for retailers through the mediating role of consumer program loyalty (Vieira et al., 2022). Jiang et al. explore the management of sales agents and product returns while implementing safeguards against fake orders (Jiang et al., 2025). Aprisadianti and Dwiyantri develop an application to detect promotion abuse fraud utilizing a risk scoring methodology (Aprisadianti and Dwiyantri, 2023). Li et al. introduce PromoGuardian, a graph based fraud detection system deployed at scale, which targets “stocking up” and cashback abuse patterns that are hard to distinguish from normal purchasing (Li et al., 2025). Sun et al. show that concession abuse (repeated exploitation of customer service credits, refunds, and goodwill gestures) has been industrialized as “Concession Abuse as a Service,” with specialized communities sharing scripts and tactics (Sun et al., 2021).

2.4. Business logic vulnerabilities

Application business logic vulnerabilities arise when an attacker can use a system exactly as designed at the interface level, but in a sequence or combination of actions that violate the underlying business rules. Unlike classic bugs such as SQL injection or buffer overflows, logic vulnerabilities involve no malformed input or protocol violation. Instead, they exploit gaps in how developers modeled the system’s state machine: missing transitions, unchecked assumptions, or corner cases that product teams did not anticipate.

Standards bodies and security communities have recognized business logic issues for over a decade. MITRE’s CWE-841, Improper Enforcement of Behavioral Workflow, covers cases where a product supports multi step workflows but fails to ensure correct ordering of steps or correct state updates on transitions (MITRE Corporation, 2026). In 2025, OWASP released the first Business Logic Abuse Top 10, which uses a Turing machine inspired model to categorize business logic vulnerabilities into ten classes including lifecycle and orphaned transition flaws, sequential state bypass, and transition validation flaws (OWASP Foundation, 2026). This release signals growing recognition that business logic abuse constitutes a distinct and expanding threat category.

Research has proposed various techniques to discover such flaws. Felmetzger et al. proposed one of the first automated approaches to detect logic vulnerabilities by inferring expected workflows from dynamic execution (Felmetzger et al., 2010). Pellegrino and Balzarotti demonstrated a black box technique that learns navigation graphs from network traces (Pellegrino and Balzarotti, 2014). Chen et al. examined logic vulnerabilities in payment syndication services, showing how inconsistencies between documentation and implementation can produce exploitable behavior (Chen et al., 2019). Wang et al. analyzed Cashier as a Service web stores and showed that integration errors can allow attackers to manipulate prices or obtain goods without paying (Wang et al., 2011). Ghorbansadeh and Shahriari proposed detecting

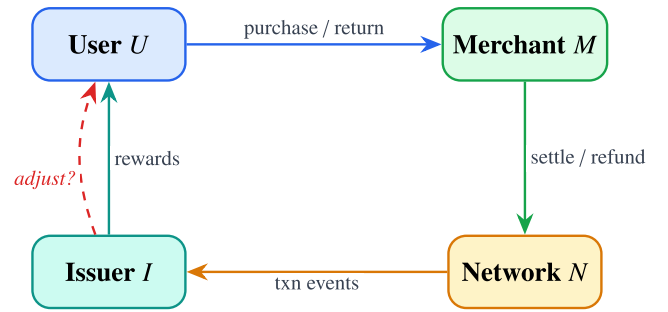


Fig. 1. Reward ecosystem entities and event flows.

application logic vulnerabilities by finding incompatibilities between design and implementation (Ghorbansadeh and Shahriari, 2020).

Our work differs from these prior strands in two key respects. First, we do not focus on collusive schemes or large scale fake orders: our adversary model considers single, honest but opportunistic consumers who exploit legitimate purchases and refunds. Second, rather than building another detection system, we focus on the internal reward engine logic itself and show that small design flaws are sufficient to create reward abuse attack opportunities that violate basic integrity invariants.

3. System and threat model

We now describe the environment in which reward engines operate and the type of adversary we consider. Our goal is to define a minimal model that is rich enough to express the case studies in Section 5 and the invariants in Section 4.

3.1. System entities

We model a generic reward ecosystem with the following entities:

- **User U** : a legitimate consumer who participates in one or more card or loyalty programs. A single user may hold multiple cards and maintain separate reward ledgers with different issuers.
- **Issuer I** : a bank, fintech company, or wallet provider that issues cards or maintains a reward ledger for users. Issuers define earning rules, redemption channels, and caps, and are responsible for implementing the reward engine logic we study.
- **Merchant M** : a retailer or service provider that accepts payments. A merchant can appear as a physical point of sale, an e commerce site, or a marketplace platform.
- **Payment Network N** : card networks and processors that handle authorization, clearing, and settlement. We treat N as honest and reliable; it determines which transactions and refunds are visible to each issuer.

Fig. 1 illustrates the relationships among these entities and the flow of transaction, refund, and reward events. Solid arrows denote the primary transaction and reward flows, and the dashed arrow denotes the reward adjustment path, which is absent in Case I and absent for cross cycle refunds in Case II.

Issuers maintain a reward ledger $B[U]$ for each user, representing cashback or points that may be convertible to cash, statement credit, or vouchers. We refer to the logic that maps observed events (purchases, refunds) to updates on this ledger as the *reward engine*.

3.2. Reward engine abstraction

We abstract rewards as simple functions over transactions and events, intentionally omitting implementation details such as database schemas or APIs. We model a reward program as a finite set of *earning rules*. A rule j carries a rate ρ_j , an eligibility predicate, a cap window function W_j mapping an accounting period to the window governing the cap, and a cap that may be unbounded, written $\text{Cap}(u, j)$ for user u under rule j in the algorithms of Section 6. Because the window belongs to the rule rather than to the program, a cap assessed per billing cycle, per quarter, or per year is expressed by one mechanism, and an uncapped rule never consults it. The cap bounds a per rule counter over that window, accumulating granted reward where the program caps reward value and eligible spend where it caps the qualifying purchase base. A transaction binds at settlement to the first rule it satisfies, and that binding is immutable. This covers flat uncapped rates, category rates capped per cycle, rotating rates capped per quarter, and tiered structures over disjoint categories. Where several rules apply to one transaction, such as a bonus rate up to a cap with a base rate beyond it, the transaction is decomposed into components, each bound to one rule with its own reward and refund accounting, and the algorithms of Section 6 apply per component.

Each **transaction** t is a tuple:

$$t = (t.id, t.u, t.m, t.a, t.c, t.n_s, t.status),$$

where $t.u$ is the user, $t.m$ the merchant, $t.a$ the amount, $t.c$ a category label (e.g., GROCERY, GAS), $t.n_s$ the billing period in which the transaction settles, and

$t.status \in \{\text{PENDING}, \text{SETTLED}, \text{PART_REF}, \text{REFUNDED}, \text{CHARGEBACK}\}.$

We assume that once a transaction reaches SETTLED, its amount and category do not change; refunds and chargebacks are modeled as separate events attached to t .

For each transaction, we maintain a scalar $R[t.id]$ that records the total reward value granted for that transaction so far. In defensive implementations, $R[t.id]$ may decrease over time as refunds are processed.

A **reward engine** implements three core operations:

- **RewardRate(j)**: the reward rate ρ_j of the earning rule j to which a transaction is bound. Eligibility, promotional multipliers, and caps are represented separately, the last through Cap and CapUsed in Algorithm 1.
- **OnSettled(t)**: invoked when a transaction becomes settled and reward eligible. This is where rewards are initially computed and credited.
- **OnRefund(t, x)**: invoked when an amount x of transaction t is refunded or charged back. This is where previously granted rewards should be adjusted.

The reward balance $B[U]$ is a numeric ledger that accumulates contributions across all settled transactions and redemptions. We explicitly allow $B[U]$ to become negative in some designs, which is essential for safely handling late refunds after rewards have been redeemed.

3.3. Event timelines

Transactions and promotions unfold over time. For a single purchase as seen by an issuer, a typical timeline proceeds through five stages:

1. At time t_{auth} , the user presents a card or wallet; an authorization event is processed by N .
2. At time t_{settle} , a settlement event with final amount $t.a$ and merchant $t.m$ is delivered to I .

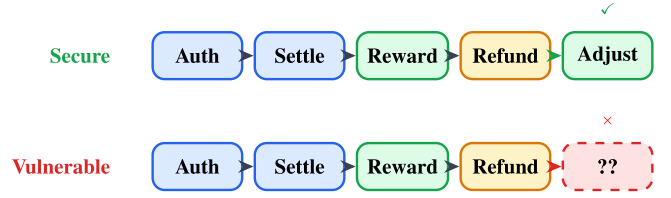


Fig. 2. Transaction reward lifecycle in a secure system (top) and a vulnerable system (bottom), for the V1 class (Case I).

3. At time t_{reward} , the issuer's reward engine invokes $\text{OnSettled}(t)$, updating $B[U]$ and $R[t.id]$. In some designs, rewards become immediately redeemable; in others, they remain pending until a statement cycle ends.
4. At some later time t_{refund} , the user may return goods; M issues a refund that flows through N to I .
5. At time t_{adjust} , if implemented, the issuer invokes $\text{OnRefund}(t, x)$, adjusting $B[U]$ and $R[t.id]$ accordingly.

Fig. 2 contrasts the transaction reward lifecycle under a vulnerable system versus a secure system. The critical distinction is the presence or absence of step 5. In secure systems, refund events trigger proportional reward adjustment, whereas in vulnerable systems step 5 is absent entirely. In V2 (Case II) the step exists within the billing cycle but not across cycle boundaries, as shown in Fig. 4, and in V3 (Issuer F) it fires with a timing asymmetry that Section 5.3.2 details.

3.4. Adversary model

We consider an honest but opportunistic *reward abuse attacker* whose capabilities mirror those of a typical consumer, with a bias toward systematic exploitation of reward programs. The adversary can initiate legitimate purchases and refunds via normal merchant channels, including full and partial returns where the merchant's policy allows. The adversary can choose among different payment instruments such as different cards or wallets. Critically, the adversary can time actions strategically: redeeming rewards as soon as they become available and then waiting to request refunds, or scheduling returns to fall in a different billing cycle than the original purchase. The adversary may hold multiple legitimate accounts with different providers, subject to normal onboarding checks. Our model assumes at most one account per issuer; multiple same issuer accounts scale the financial impact linearly but do not change the fundamental vulnerability class or the correctness argument for the defensive algorithms.

The adversary does *not* compromise backend systems at issuers or merchants, forge or tamper with payment network messages, use stolen cards or identities, make false claims about non receipt or product condition, or exploit low level implementation bugs such as SQL injection or memory corruption. Their goal is to increase net financial gain from rewards beyond what program rules intend, specifically to obtain net rewards inconsistent with net spend, while minimizing detection and account closure risk.

We note that prior work documents industrialized, scripted exploitation of concession and refund workflows (Sun et al., 2021), suggesting that automated adversaries operating at higher velocity are realistic. Our adversary model focuses on the single user manual tier because it represents the minimum capability needed to exploit the vulnerabilities we describe. Importantly, the defensive algorithms we propose in Section 6 operate as event driven controls over persistent transaction and ledger state (per transaction reward tracking, proportional clawback on each refund event, negative balance enforcement on each redemption request). These mechanisms function identically regardless of whether the refund events arrive from a single manual user or from an automated script generating hundreds of refund requests per hour.

3.5. Security properties

We focus on two main security properties. **Reward Integrity** requires that the total economic value a user derives from rewards should never exceed what they are entitled to based on their net legitimate spending and published program rules. **Refund Reward Consistency** requires that when purchases are reversed via refunds or chargebacks, the reward state (balances, per transaction records) should be updated within a bounded time to remain consistent with the new economic reality. These are formalized in Section 4. We do not aim to model every form of fraud; instead, we focus on design flaws in reward engines that even a single honest but opportunistic user can exploit. We treat higher level fraud analytics and anomaly detection systems as orthogonal: they may detect or throttle some abuse patterns, but they cannot restore correctness if the core reward logic violates basic invariants.

4. Reward logic invariants and vulnerability classes

We now formalize what it means for a reward system to behave correctly with respect to user spend and refunds, and classify the main ways in which real world systems violate these properties.

4.1. Formal definitions

Let $\mathcal{T}(U)$ denote the set of all transactions involving user U as observed by a particular reward engine.

Net spend. For each $t \in \mathcal{T}(U)$, let x_t denote the total amount refunded or charged back for that transaction (possibly 0). We define the *net spend* of user U as:

$$\text{NetSpend}(U) = \sum_{t \in \mathcal{T}_{\text{settled}}(U)} (t.a - x_t) \quad (1)$$

where $\mathcal{T}_{\text{settled}}(U)$ denotes all transactions that have reached at least SETTLED status (including PART_REF, REFUNDED, and CHARGEBACK, all of which were settled before being reversed), and $x_t = \text{TotalRefunded}[t.id]$ is the cumulative refund amount as of the evaluation time. Eq. (1) is a snapshot quantity: for a PART_REF transaction, x_t reflects the refunds processed so far and may increase as additional partial refunds arrive. For a fully refunded transaction, $x_t = t.a$ and the contribution is zero. Because each earning rule carries its own rate and its own cap window, we also define the net spend restricted to a (rule, window) bucket,

$$\text{NetSpend}_{j,w}(U) = \sum_{\substack{t \in \mathcal{T}_{\text{settled}}(U) \\ \text{Bound}[t.id] = (j,w)}} (t.a - x_t) \quad (2)$$

so that $\text{NetSpend}(U) = \sum_{j,w} \text{NetSpend}_{j,w}(U)$.

Net reward. Let $B[U]$ be the current reward balance and let $R_{\text{red}}(U)$ be the total value of rewards already redeemed. We define:

$$\text{NetReward}(U) = B[U] + R_{\text{red}}(U) \quad (3)$$

Reward integrity. A reward system satisfies *Reward Integrity* if, for all users U :

$$\text{NetReward}(U) \leq f(\text{NetSpend}(U), \text{PromoRules}) \quad (4)$$

where $f(\cdot)$ encodes the program's published reward rates, the cap on each earning rule over that rule's own accounting window, and promotions. For a simple 1 percent cashback program with a single category, $f(s, \cdot) = 0.01 \cdot \max(s, 0)$. For programs with per category rates and per period caps, f is not a function of scalar net spend. It decomposes over (rule, window) buckets as $f = \sum_{j,w} \min(\lfloor \rho_j \cdot \text{NetSpend}_{j,w}(U) \rfloor, \text{Cap}(u, j))$, where $\text{NetSpend}_{j,w}(U)$ is the net spend of user U bound to rule j within that rule's cap window w , evaluated in integer minor units. The bucket index is the rule and its own window rather than a category and a billing period, so a rule capped per quarter or per year is bounded

once per quarter or per year rather than once per statement. We write $f(\text{NetSpend}(U), \text{PromoRules})$ as shorthand for this decomposition throughout. Case I is precisely a violation: refunds drive $\text{NetSpend}(U)$ toward zero, but $\text{NetReward}(U)$ remains positive.

Refund reward consistency. A reward system satisfies *Refund Reward Consistency* if, for any transaction t of user U , whenever a nonzero refund amount $x_t > 0$ is processed, there exists a bounded time Δ such that after at most Δ time:

1. the state of t reflects the cumulative refunded principal, so that $t.\text{status}$ is PART_REF when $0 < x_t < t.a$ and REFUNDED or CHARGEBACK when $x_t = t.a$, and
2. the reward state has been updated so that condition (4) still holds.

This allows for asynchronous or batched processing but rules out systems where rewards are never adjusted. Both properties are defined over events the reward engine has observed. A refund that is economically effective at the merchant but has not yet reached the issuer cannot be reconciled, so the interval between merchant action and event receipt is an operational exposure rather than a violation of the invariant. Live event ingestion shortens that interval, and a redemption hold prevents value from leaving the ledger during it.

Relationship between the two invariants. Reward Integrity and Refund Reward Consistency play different roles and are independently necessary. Reward Integrity is a safety property. Evaluated at a quiescent point, after all delivered refunds have been processed, it bounds net reward by entitlement through inequality (4). Refund Reward Consistency is a bounded liveness property. After any refund it requires the system to return to an Integrity satisfying state within bounded time Δ . Neither implies the other. A system can satisfy Reward Integrity at isolated snapshots yet violate Consistency by restoring the invariant only after an unbounded delay, during which redeemable value can be extracted, while a synchronous engine with $\Delta = 0$ satisfies Consistency trivially. This separation distinguishes the classes of Section 4.3. V1 violates both, since no restoration ever occurs; V2's defining failure is Refund Reward Consistency for post boundary refunds, with the Reward Integrity violation following as a consequence; and V3(b) meets Consistency within one billing cycle but violates Integrity transiently. Table 1 therefore lists the defining invariant per class, whereas Table 2 marks Issuer B as violating both, since its Consistency failure induces a downstream Integrity violation.

4.2. Additional invariants

Implementations may enforce further properties:

- **Per Transaction Bound.** For each transaction t , the total reward should never exceed a known function of $t.a$ and the rate of the rule to which t is bound.
- **Monotonicity with Net Spend.** If a user's net spend does not increase under any earning rule over a time window, their net reward should not increase either. The per rule qualifier is necessary because $f(\cdot)$ applies a distinct rate and cap to each rule. A user who reverses a purchase in a low rate category and makes an equivalent purchase in a high rate category leaves scalar net spend unchanged while increasing net reward, and this outcome is correct rather than anomalous. Our algorithms do not enforce this property directly. Reward Integrity remains the operative bound, and it continues to hold in that situation because the increased reward stays within the rate and cap of the rule under which the new spend occurred.

Table 1

Vulnerability classes as the outcome of three successive questions, with defining invariant, OWASP alignment, and instantiating case. RI = Reward Integrity, RRC = Refund Reward Consistency.

Class	Adj. fires?	Past finalization?	Redeem gated?	Invariant	OWASP BLA	Case
V1	No	–	–	RI, RRC	Class 1	I
V2	Yes	No	–	RRC	Class 7	II
V3(b)	Yes	Yes	No	RI (transient)	Class 4	III-B
Robust	Yes	Yes	Yes	none	–	III-A

Table 2

Comparative issuer behavior across reward dimensions. ✓ = satisfied; × = violated; ~ = partially satisfied.

Property	A	B	C	D	E	F
Reward timing	Instant	Stmt. close	Instant	Stmt. close	Stmt. close	Instant
Refund adjustment	None	Cycle scoped	Immediate	Stmt. close	Stmt. close	Stmt. close
Neg. balance support	N/A	N/A	Indefinite	Indefinite	Indefinite	Indefinite ^a
Reward integrity	×	×	✓	✓	✓	~
Refund reward consistency	×	×	✓	✓	✓	✓

^a Recovery depends on future qualifying spend, which the user can unilaterally withhold. “Cycle scoped” denotes adjustment within the open cycle only.

4.3. Vulnerability taxonomy

From our modeling and case studies, we identify three primary classes of reward logic flaws relevant to refund based reward abuse:

- V1: Refund Insensitive Rewards.** Rewards are granted and become redeemable, but are never adjusted when the underlying transaction is refunded. The reward engine has no refund event handler or refund aware code path. The reward ledger behaves as if only positive settlement events exist. This directly violates both Reward Integrity and Refund Reward Consistency (Case I).
- V2: Temporally Scoped Adjustment.** The reward engine includes a refund aware computation mechanism, but that mechanism operates only up to a *finalization boundary*, the point at which the engine treats a period’s reward as settled and no longer subject to adjustment. Refunds arriving before the boundary are folded into the computation; refunds arriving after it are never processed, so they produce no clawback and record no reward debt. The class is defined by the existence of such a boundary, not by where a given engine places it. In Case II the boundary is statement close, and a cross cycle refund is simply how a post boundary refund presents in that architecture. This differs from V1 in that a pre boundary adjustment mechanism exists; it differs from V3 in that post boundary refunds produce zero clawback rather than a clawback that is computed but not enforced against redemption. This is structurally distinct from TOCTOU, which involves concurrency; V2 arises from a deterministic architectural choice to treat each period as final once the boundary passes, leaving any later refund unreachable regardless of when the merchant permits it. Two factors are explanatory rather than definitional. The length of the merchant’s return window governs how often refunds arrive after the boundary but does not create it, and an engine without persistent per transaction linkage cannot compute a post boundary clawback at all, which is a common cause of V2 rather than a defining property, since an engine that maintains linkage may still decline to revisit a closed period. This maps to CWE-841 (Case II).
- V3: Inconsistent Negative Balance Handling.** A refund creates a reward debt, but the system does not reliably prevent the user from extracting value against that debt. V3 manifests in two forms: (a) no negative balance support at all, where the system simply ignores the reward debt; and (b) a negative balance is correctly created but a timing asymmetry between instant reward availability and batched clawback allows the user to extract value before the adjustment occurs, and recovery depends entirely on continued card usage that the user can unilaterally withhold. Form (a) is included for taxonomic completeness; we did not observe a pure

V3(a) instance in our sample, though it is architecturally plausible for systems that cap negative balances at zero. V3(a) is structurally distinct from V1: in V3(a), an OnRefund handler exists and fires correctly, but the balance update discards the computed debt by flooring at zero rather than permitting a negative balance. The observable outcome after redemption is identical to V1, but the root cause and required fix differ: V1 requires building a handler; V3(a) requires only removing the zero floor guard. The two are nonetheless separable before redemption. A flooring implementation still debits an available positive balance, so an unredeemed balance left unchanged by a refund indicates an absent handler rather than a discarded debt. Case I exhibits that behavior and is classified V1 on that basis. Because we did not observe a V3(a) instance, it is retained here as a completeness note and is not listed as a separate row in the class summary of Table 1. Case III illustrates both robust and partially robust approaches.

V2 and V3(b) are separated by which invariant fails and which repair applies, not by severity. V2 is a coverage defect in the adjustment path: an adjustment mechanism exists but its scope does not reach past the finalization boundary, so post boundary refunds yield no clawback and record no reward debt. Its defining failure is Refund Reward Consistency, remedied by extending adjustment scope or adding per transaction linkage (Algorithm 4). V3(b) is a gating defect in the redemption path: the adjustment fires and records the negative entry on schedule, but redemption is not gated on it having settled, which transiently violates Reward Integrity while restoration still completes within one cycle. It is remedied by gating redemption on clawback settlement (Algorithm 3, guard a). The difference in broken invariant and required repair places the two in separate classes.

Table 1 states the classes as the outcome of three successive questions about the reward engine, which makes their separation explicit. The first three columns are the discriminating questions, and no two instantiated classes share the same pattern of answers, so the classes are distinguished by construction rather than by assertion. The table classifies the dominant mechanism we observed in each case; a deployed engine could in principle combine failure modes, for example omitting some refund paths while temporally scoping others. Each row also records the invariant that fails and, at the branch where it fails, the remedy that applies. V1 requires building a refund handler (Algorithms 1 and 2); V2 requires retroactive cross cycle reconciliation (Algorithm 4), which first presupposes per transaction linkage; and V3(b) requires gating redemption on clawback settlement (Algorithm 3, guard a). The table lists only classes we instantiated in our case studies; the V3(a) form discussed above is a completeness note rather than an observed instance and is therefore omitted here.

All three vulnerability classes map to CWE-841 (improper enforcement of behavioral workflow) under parent category CWE-840 (Business Logic Errors), as the system fails to enforce the required sequence of: purchase → reward → refund → reward adjustment (MITRE Corporation, 2026). In the OWASP Business Logic Abuse Top 10, V1 aligns with Class 1 (Lifecycle & Orphaned Transition Flaws), where the refund to reward adjustment transition is entirely absent; V2 aligns with Class 7 (Transition Validation Flaws), where a within-scope adjustment mechanism exists but its temporal coverage is insufficient to handle cross-cycle refunds; V3(b) aligns with Class 4 (Sequential State Bypass), where the redemption step proceeds before the required clawback settlement has completed (OWASP Foundation, 2026).

5. Case studies

We now present three case studies that instantiate the vulnerability classes above. In each case we used small, legitimate transactions on accounts we personally control and followed normal user workflows. All providers are anonymized.

5.1. Case I: Refund insensitive cashback (Issuer A)

Context. Issuer A operates a debit based cashback program in which users earn 5 percent cashback on purchases in selected categories, subject to a monthly cap equivalent to approximately \$50 per month. Users can enable a specific category each month (for example, groceries, restaurants, or general retail). Rewards are credited shortly after the transaction posts and become immediately redeemable as a cash equivalent balance that can be transferred to any linked account.

Observed behavior. Using a personal account, we performed low value test purchases at merchants within the 5 percent category. We waited until the cashback appeared and became available for redemption, which typically occurred within minutes to hours of settlement. We then redeemed the earned rewards by converting them to cash equivalent balance and transferred the funds to another account. Finally, we initiated legitimate refunds through standard merchant return processes, including both full and partial returns where the merchant's policy allowed.

Across the merchants tested, when the refund posted to the account, the principal amount was correctly returned to the debit balance. However, no adjustment was made to the reward ledger: previously granted rewards were neither debited nor marked as pending reversal. Where a positive unredeemed balance remained at the moment the refund posted, that balance was likewise unchanged, and the ledger was never driven negative. This separates an absent adjustment path from one that computes a clawback and floors the result at zero, since a flooring implementation would still debit an available positive balance. Future rewards accrued normally and were completely unaffected by the refunded transactions. After a full refund, the user's net spend on those purchases returned to approximately zero, while net rewards remained positive. This constitutes a direct violation of Reward Integrity.

Four phase attack workflow. The attack follows a deterministic four phase pattern, illustrated in Fig. 3. In the first phase (*Earn Rewards*), the attacker purchases items in the bonus category and waits for the cashback to post, typically within minutes to hours of settlement. In the second phase (*Extract Cash*), the attacker redeems the accrued cashback into a spendable balance and transfers it to an external account, removing the value from the reward ledger's reach. In the third phase (*Return & Refund*), the attacker returns the purchased items through the merchant's standard return process and receives a refund of the principal amount to the card. In the fourth phase (*Retain Rewards*), the attacker observes that the reward ledger has not been adjusted: the net spend has returned to approximately zero, but the redeemed cashback remains in the external account. Because the reward engine does not process refund events, the cycle can be repeated each billing period up to the monthly reward cap.

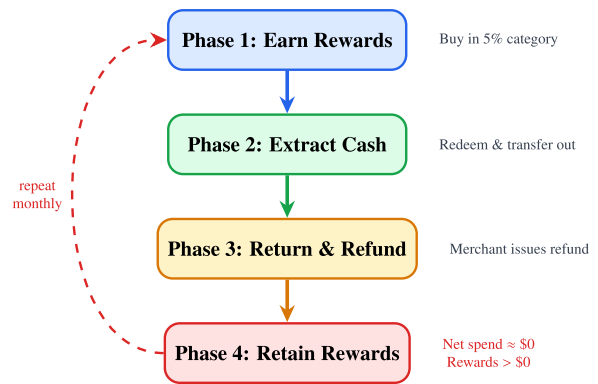


Fig. 3. The four phase double dip cashback reward abuse attack (DDRA) workflow for Case I.

Classification. This vulnerability falls under CWE-841 (Improper Enforcement of Behavioral Workflow) within the CWE-840 Business Logic Errors category. The expected workflow “Purchase → RewardGrant → Refund → RewardAdjust” is not enforced: the RewardAdjust step is entirely absent. In the OWASP Business Logic Abuse Top 10, this maps to Class 1 (Lifecycle & Orphaned Transition Flaws), as the refund to reward adjustment transition is missing entirely (OWASP Foundation, 2026). We note that the issuer's own published program terms reserve a right to reduce or remove rewards on returns, refunds, and chargebacks, including rewards already redeemed. The missing transition is therefore an unenforced instance of the issuer's stated policy rather than an absence of any clawback intent, which we discuss further in Section 7.3.

Impact estimation. The program cap provides a theoretical ceiling on retained reward. Under successful qualifying purchases, redemptions, and refunds, a single user retains at most the monthly cap each billing period, which for a \$50 cap is at most \$600 per user per year. This is an accounting upper bound rather than a measured attacker profit, since realized exploitation depends on merchant return policies, fees, return friction, issuer monitoring, and account action, none of which we measured. Program wide exposure is the product of this ceiling, the active population $|U|$, and the fraction p of users who exploit the behavior. We do not estimate p , and we present the population scaled figures in Section 7.4 only as a sensitivity range across assumed values of p , not as a prediction of realized losses.

Case I Takeaway. Refund insensitive cashback accounting allows users to redeem rewards on purchases that are later fully refunded, creating a deterministic double dip attack using legitimate accounts and standard refund workflows.

5.2. Case II: Statement cycle timing exploit (Issuer B)

Context. Issuer B operates a credit card program with a statement cycle based reward computation model. During an open billing cycle, each transaction displays a provisional reward entry that reflects its current state, positive for a settled purchase and reduced or negative for a refund on a same cycle transaction, but no reward is redeemable until the cycle closes. At statement close, the issuer computes and posts rewards over the net of refund base for that period and makes the resulting cashback available, in this account through an auto redemption feature that converts the posted rewards into statement credits. Same cycle refunds are therefore folded into the computation before rewards are finalized. A refund on a transaction from an already closed cycle does not alter the reward posted for that cycle.

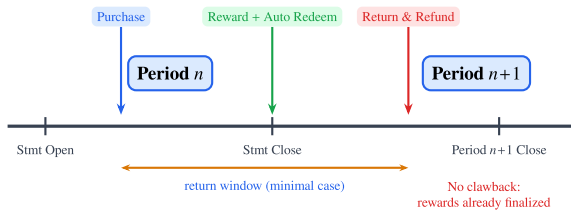


Fig. 4. Statement cycle finalization boundary in Case II, shown for the second tested transaction.

Observed behavior. Using a personal account with the auto redemption feature enabled, we tested three qualifying purchases whose refunds arrived at different points relative to statement close. Each displayed a provisional reward entry during the open cycle, with no reward redeemable until close. The first was refunded in full while its cycle was still open; its provisional entry was reduced accordingly, and at close the issuer posted the reward over the net of refund base, so no reward was posted for the refunded amount. The second and third were not refunded before close, and at the close of period n the issuer posted and auto redeemed their rewards as a statement credit. The second was then returned in the following period as a full refund, and the third several cycles later as a partial refund; both refunds posted as credits, and in neither case was the reward already posted for period n adjusted. The three transactions therefore separate the two behaviors directly: a refund before close reduces the reward eligible base and is reflected in the posted reward, whereas the two post close refunds produced none. Because those two differ in both delay (one cycle versus several) and magnitude (full versus partial), the absence of adjustment is attributable to the refund's position relative to close rather than to its size or to a bounded reconciliation delay. Longer horizons and other channels were not enumerated, so the classification rests on the observed cycle scoped adjustment rather than on exhaustive enumeration.

Fig. 4 illustrates the finalization boundary for the second tested transaction, whose refund lands in the period following statement close. The third differs only in that its refund arrived several cycles later and was partial.

Classification. This is an instance of V2 (Temporally Scoped Adjustment). Issuer B's reward engine does perform refund aware computation, but its scope is bounded to the open billing cycle. The first tested transaction gives direct evidence of this scope, since its refund arrived before close, reduced the reward eligible base, and was reflected in the reward posted for that period. What the engine lacks is any mechanism to revisit a period once it has closed, so a refund on a prior cycle transaction produces no adjustment. Within cycle adjustment separates Issuer B from Issuer A. Issuer A performs no refund adjustment at any time (V1), whereas Issuer B adjusts within the open cycle but not across a closed boundary (V2). The operative boundary is statement finalization rather than the length of any merchant return window. Statement finalization instantiates, for this architecture, the finalization boundary of Section 4.3, and a refund falling after that boundary receives no adjustment regardless of its magnitude or timing. This differs from V1 (Case I), where the reward engine has no refund awareness at all, and from V3 (Case III-B), where the adjustment mechanism reaches past finalization but redemption is not gated on it having settled.

Impact. The defect is independent of the earning rate: because no clawback is computed once a period has closed, the retained reward is whatever the engine posted, whether at a promotional category rate or a base rate on general spend, so the exposure extends to all reward earning purchases rather than to promotional categories alone. The behavior is consistent with a design lacking, or not using, persistent

per transaction linkage for post close clawback, since the observable reward computation is tied to the billing cycle rather than to individual transactions. Per user exposure follows the cap structure: where a category is capped, extraction per cycle is bounded by that cap, as in Case I; where a tier is uncapped, it is bounded instead by the reward earning spend the user can reverse within a cycle. We do not quantify the uncapped regime, which would require assumptions about credit limits and return friction that we did not measure.

Case II Takeaway. Statement cycle reward computation nets same cycle refunds before rewards are posted, but treats each period as final once its statement closes, so a refund on a transaction from a closed period receives no adjustment. Adjustment is therefore scoped to the open billing cycle, producing a reward integrity violation through a different mechanism than Case I.

5.3. Case III: Negative balance designs (Issuers C, D, E, and F)

To contrast with Cases I and II, we examined credit card reward programs from four additional issuers. Our observations reveal a spectrum of negative balance handling, ranging from indefinite enforcement to a partially robust design with a bounded recovery window.

5.3.1. Case III-A: Robust indefinite enforcement (Issuers C, D, and E)

Context. Issuers C, D, and E each operate credit card reward programs whose public documentation and observed behavior indicate robust refund handling. All three issuers state that rewards are computed on net purchases and may be reversed on returns, even resulting in negative reward balances.

Observed behavior. Using personal accounts and small transactions, we observed the core negative balance enforcement pattern at all three issuers. A purchase in a rewarded category posts and yields a positive reward increment. If the user redeems that reward (for example, as a statement credit), the reward balance reflects the redemption. When a refund later posts for the original transaction, the issuer debits the corresponding rewards, sometimes resulting in a negative balance if the user had already redeemed. The negative balance persists indefinitely: subsequent purchases first reduce the negative balance, and only after returning to a non negative balance does the user accumulate redeemable rewards again.

While all three issuers enforce the same indefinite negative balance netting outcome, they differ in their reward timing architecture. Issuer C posts reward points shortly after transaction settlement and applies clawback immediately upon refund processing. Issuers D and E compute and post reward points at statement cycle close, and apply refund adjustments at the same cadence. Despite this architectural difference, the security relevant behavior is identical: in all three cases we observed no forgiveness or time bounding of the negative balance, and future earnings were automatically offset against the outstanding debt across every billing cycle we tracked. Under the workflows we tested, this pattern was consistent with Reward Integrity.

Case III-A Takeaway. Issuers that link rewards to individual transactions, apply clawback consistent with proportional adjustment in the tested refund workflows, and enforce negative balances indefinitely against future earnings provide robust protection against refund based reward abuse. Three production issuers exhibited this pattern under two different reward timing architectures, instant and statement cycle, which indicates that it is workable under both.

5.3.2. Case III-B: Timing asymmetry in reward clawback (Issuer F)

Context. Issuer F operates a credit card reward program in which reward points become available for redemption as soon as the underlying transaction settles (typically within three business days of purchase). However, reward adjustments for refunds are not processed until statement close. This architectural choice creates a timing asymmetry: the user can access and redeem reward value in near real time, but the system does not reconcile refund based clawbacks until the end of the billing cycle.

Observed behavior. Using a personal account, we confirmed this asymmetry through a controlled sequence. We made a purchase in a rewarded category and observed that the reward points appeared in the redeemable balance within days of settlement, well before the current billing cycle closed. We redeemed the full reward balance as a statement credit. We then initiated a return through the merchant's standard process and received a refund on the original transaction. The refund posted to the card account promptly, but the reward ledger was not adjusted until the next statement close. At that point, Issuer F correctly created a negative reward entry, debiting the ledger by the amount corresponding to the refunded transaction. The negative balance persisted: subsequent reward accruals first offset the negative balance before becoming redeemable again. The critical observation is the *window* between reward redemption and clawback. Because points are redeemable immediately upon settlement but clawback occurs only at statement close, a user who redeems rewards and then initiates a refund within the same billing cycle extracts value that will not be debited until the following statement close. Card issuers generally do not permit account closure while any outstanding obligation remains, so the negative balance cannot be escaped through account termination. However, a user who redeems, refunds, and then simply stops making qualifying purchases on the card can sustain a negative reward balance indefinitely, so that recovery through the observed future accrual offset may be delayed indefinitely; other recovery mechanisms were not assessed. The system eventually converges toward Reward Integrity only if the user continues normal card activity.

Classification. This behavior is an instance of V3 (Inconsistent Negative Balance Handling), specifically the V3(b) sub form where the timing asymmetry between instant reward availability and batched clawback creates an exploitable window. The `ONREFUND` handler fires correctly and the negative balance entry is created on schedule; the workflow violation occurs at the redemption gate, which does not enforce that outstanding clawback has settled before permitting redemption. This maps to CWE-841 (Improper Enforcement of Behavioral Workflow), as the required sequence of clawback settlement prior to redemption eligibility is not enforced (MITRE Corporation, 2026). In the OWASP Business Logic Abuse Top 10, this aligns with Class 4 (Sequential State Bypass) (OWASP Foundation, 2026).

Impact. The financial exposure per incident is smaller than in Cases I and II because the negative entry does eventually offset future rewards during normal continued use. The exploit is not a permanent one time extraction (as in Cases I and II where no clawback ever occurs) but rather a timing based extraction where the user retains reward value that remains unrecouped for one or more billing cycles. The issuer bears the cost of this float and the risk of non recovery if the user reduces or ceases card activity.

Case III-B Takeaway. Creating a negative reward balance is necessary but not sufficient. When reward availability is instant but clawback is batched to statement close, the timing asymmetry enables users to extract value before the adjustment occurs. Robust designs (Case III-A, Issuer C) eliminate this gap by processing clawbacks immediately upon refund settlement.

Three tier summary. Table 2 summarizes the comparative behavior across all six issuers studied, ordered A through F. The results reveal a three tier spectrum: vulnerable (Issuers A and B, where refunds do not trigger cross cycle reward clawback), robust in the tested workflows (Issuers C, D, and E, indefinite negative balance enforcement), and partially robust (Issuer F, where rewards are available instantly but clawback is deferred to statement close, creating a timing based escape path). The vulnerable and partially robust cases also differ in recoverability. In Cases I and II we observed no adjustment and no mechanism by which the value would be recovered, whereas in Case III-B a negative entry is created and offset by subsequent qualifying rewards, making recovery conditional on continued activity the user may withhold rather than automatic. Issuer F's marks in Table 2 follow from this. Restoration within one billing cycle is a bounded delay, so Refund Reward Consistency is met, whereas Reward Integrity is violated transiently during the window between redemption and clawback.

6. Formal model and defensive algorithms

We now present the formal state machine model and a suite of defensive pseudo algorithms. Our goal is not to prescribe a specific implementation, but to capture design patterns that enforce Reward Integrity and Refund Reward Consistency. While a full machine checked specification is beyond the scope of this paper, the four algorithms and four lemmas constitute an invariant preserving design pattern, argued correct under the stated assumptions and scope, that practitioners can adapt to their specific execution environments.

Transaction reward state machine. We model each transaction t and its associated reward state as a pair $(t.status, R[t.id])$. The full system state consists of transaction states (PENDING, SETTLED, PART_REF, REFUNDED, CHARGEBACK), a mutable per transaction reward record $R[t.id]$ holding the reward currently retained, reassigned on each clawback, and an immutable original reward record $R_{orig}[t.id]$ set once at settlement, both initialized at settlement and never left undefined, a cumulative refund counter $TotalRefunded[t.id]$ (initially 0), a settlement idempotency flag $RewardSettled[t.id]$ (initially false), a set `ProcessedRefunds` of refund event identifiers already applied, and a user level reward ledger $B[U]$ that aggregates contributions from all transactions plus redemptions. All per transaction records (R , R_{orig} , $TotalRefunded$, $RewardSettled$, `Bound`) must persist for the lifetime of the account; garbage collecting these records after N billing cycles would re-expose the system to the V1 vulnerability for late refunds. The binding record is required alongside the others because Algorithm 2 consults it to determine whether a refund falls within the cap window of the rule under which the transaction settled. Where a late refund arrives for a transaction whose records are unavailable, the engine cannot compute a proportional clawback and should escalate the event for manual reconciliation rather than discard it, holding redemption for the account in the interim. `ProcessedRefunds` is indexed by event rather than by transaction, and need only be retained for the deduplication window over which the ingestion layer may redeliver an event, so it is not subject to the same lifetime requirement. A well formed execution requires that every transition to `SETTLED` may trigger reward crediting, and every transition to `PART_REF`, `REFUNDED`, or `CHARGEBACK` must trigger a compensating adjustment. In Case I, this compensating step is entirely absent; in Case II, it exists within the billing cycle but not across cycle boundaries. The modeled lifecycle is deliberately restricted. Settlement, partial and full refunds, and redemption are modeled explicitly, duplicate delivery is handled idempotently under A2, and a refund arriving before settlement is queued by the ingestion layer and applied immediately after, which is the mechanism Algorithm 4 uses for same period refunds. We assume that layer preserves the relative order of events on a given transaction and replays queued events durably, since Algorithm 4 relies on a late refund still being

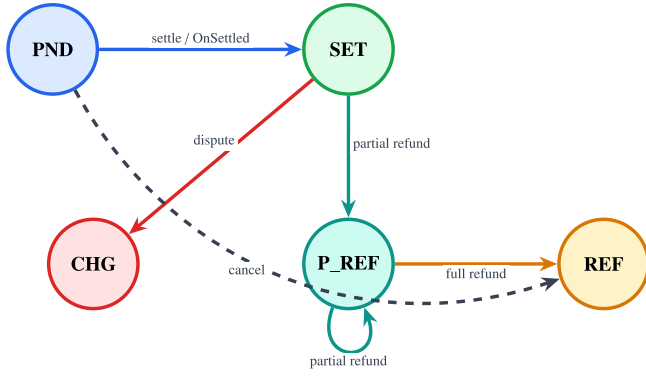


Fig. 5. Transaction reward state machine. PND = Pending, SET = Settled, P_REF = Partially Refunded, REF = Refunded, CHG = Chargeback.

Algorithm 1: RewardOnSettlement

Input: Transaction t (Section 3.2)
Output: Updated balance $B[u]$ and records $R[id]$, $R_{orig}[id]$

```

if RewardSettled[id] then
    return; // idempotency guard: A2
j ← RuleOf( $t$ ); // first rule whose predicate  $t$  satisfies
w ←  $W_j(t.n_s)$ ; // cap window of rule  $j$ 
Bound[id] ← ( $j, w$ ); // immutable after settlement
rate ← RewardRate( $j$ );
cap ← Cap( $u, j$ );
used ← CapUsed[ $u, j, w$ ];
r ← max(0, min( $\lfloor rate \cdot a \rfloor$ , cap - used)); // integer minor units
R[id] ← r;
Rorig[id] ← r; // immutable: never modified
if r > 0 then
    B[ $u$ ] ← B[ $u$ ] + r;
    CapUsed[ $u, j, w$ ] ← used + r;
    LogRewardEvent( $id, u, +r$ , "settle");
RewardSettled[id] ← true;
t.status ← SETTLED;

```

present when the next reconciliation runs. Refund reversals and corrections, chargeback reversals, reward expiration, redemption reversal, and account closure are outside the model and require compensating transitions we do not specify.

Fig. 5 illustrates the state machine with its required transitions and reward operations. Transitions to SET trigger reward crediting via Algorithm 1, and transitions to P_REF or REF trigger proportional clawback via Algorithm 2. The self loop on P_REF represents sequential partial refunds on the same transaction, the cancel transition from PND does not involve rewards, and the SET to CHG transition triggers reward reversal atomically under Assumption A5.

We now present four defensive algorithms. Within them we write id , u , and a for $t.id$, $t.u$, and $t.a$. Together, they enforce both invariants across instant reward and statement cycle architectures. Algorithm 1 handles reward crediting upon settlement. Algorithm 2 applies proportional clawback on refunds, directly closing the Case I loophole. Algorithm 3 gates redemption against balance constraints. Algorithm 4 reconciles statement cycle systems against pending refunds, addressing Case II.

Algorithm 1 computes the reward as the product of the rule rate and transaction amount, capped by the remaining cap headroom for that rule's window. The CapUsed counter is incremented by the granted

reward, ensuring that subsequent transactions in the same cap window observe the reduced cap headroom. Two per transaction records are stored: $R[id]$ tracks the current reward balance and is decremented on each partial refund, while $R_{orig}[id]$ preserves the original settlement reward as an immutable reference for proportional clawback computation. This separation is essential for correctness under sequential partial refunds, as discussed below. The status assignment on ($t.status \leftarrow$ SETTLED) is a no op when Algorithm 1 is invoked by an event driven handler on already settled transactions; it is required when invoked from Algorithm 4's Phase 2 on PENDING transactions.

Algorithm 2: RewardOnRefund

Input: Transaction t , refund amount x , refund event identifier e , accounting period n
Output: Updated balance $B[u]$ and record $R[id]$

atomic; // single transaction on the records below
if $e \in$ ProcessedRefunds **then**
 | **return;** // idempotency guard: A2
if $t.status \notin$ {SETTLED, PART_REF} **then**
 | **return;** // skip non eligible transactions
 $x_{prev} \leftarrow$ TotalRefunded[id];
if $x_{prev} + x > a$ **or** $x \leq 0$ **then** // requires A3
 | **serialization**
 | **return;** // invalid: exceeds original amount
 $x_{new} \leftarrow x_{prev} + x$;
 $R_{target} \leftarrow \left\lfloor \frac{(a - x_{new}) \cdot R_{orig}[id]}{a} \right\rfloor$; // cumulative target
 $r_{claw} \leftarrow R[id] - R_{target}$;
 $B[u] \leftarrow B[u] - r_{claw}$; // balance may go negative
 $R[id] \leftarrow R_{target}$;
 TotalRefunded[id] ← x_{new} ;
 (j, w) ← Bound[id];
if $W_j(n) = w$ // same cap window **then**
 | CapUsed[u, j, w] ← max(0, CapUsed[u, j, w] - r_{claw});
 LogRewardEvent($id, u, -r_{claw}$, "refund");
if $x_{new} = a$ **then**
 | $t.status \leftarrow$ REFUNDED;
else
 | $t.status \leftarrow$ PART_REF;
 ProcessedRefunds ← ProcessedRefunds $\cup \{e\}$;

Algorithm 2 assigns the retained reward from the cumulative refunded principal, using the immutable $R_{orig}[id]$ stored at settlement rather than the running value, which is what prevents rounding error from compounding across sequential partial refunds. The CapUsed counter is decremented only when the refund falls in the same cap window as the original settlement; because the window belongs to the rule, a rule capped per quarter restores headroom for any refund arriving in that quarter even if it falls in a later statement. For refunds arriving after the window closes, decrementing the current window's counter would create headroom never consumed within it. The negative balance created by the clawback is the primary defense against cross cycle abuse: the user cannot extract the debt, which must be repaid from future earnings before any redemption is possible. The design permits $B[u]$ to become negative for this purpose. Algorithm 2's entry guard accepts SETTLED and PART_REF but not CHARGEBACK. The exclusion is deliberate. Standard refunds are modeled explicitly, whereas chargeback outcomes invoke an equivalent compensation callback under Assumption A5, and chargeback reversals lie outside the present model. The amount guard is likewise a validity check rather than a discard rule. A refund whose cumulative principal would exceed $t.a$, or whose amount is non positive, indicates a malformed or mis-canonicalized event, and the engine should escalate it for manual reconciliation rather than drop it silently, holding redemption for the

Table 3

Sequential partial refunds on a \$100.00 purchase at a 5% rate, in integer minor units.

Event	R_{orig}	$R[id]$	$B[u]$	Status	Entitl.	Running
Settle \$100.00	500	500	500	SET	500.00	500
Refund \$33.33	500	333	333	P_REF	333.35	334
Refund \$33.33	500	166	166	P_REF	166.70	168
Refund \$33.34	500	0	0	REF	0.00	0

Algorithm 3: AuthorizeAndRedeem

Input: User u , requested redemption amount y
Output: true if the redemption was performed, else false
atomic ; // single transaction, row lock, or compare and swap on $B[u]$
begin
 if $\text{CurrentDate}() < \text{RedemptionHoldUntil}[u]$ **then**
 return false ; // grace period active
 if $B[u] - y < B_{\min}$ **then**
 return false ; // insufficient balance
 $B[u] \leftarrow B[u] - y$;
 $R_{\text{red}}(u) \leftarrow R_{\text{red}}(u) + y$;
 $\text{LogRedemption}(u, y)$;
 return true;

account in the interim as it does for a transaction whose records are unavailable.

Table 3 traces Algorithm 2 on sequential partial refunds that do not divide evenly, in integer minor units for a single transaction with $B[u] = 0$ initially and no redemption. The entitlement column is the unrounded proportional value $R_{\text{orig}}[id](a - x_t)/a$, and the running column recomputes each clawback from the running retained reward rather than from the immutable $R_{\text{orig}}[id]$. That variant exceeds entitlement at both intermediate states and the excess grows across refunds, whereas the cumulative target of Lemma 2 stays at or below entitlement throughout. Both reach zero on full reversal, so the difference is a transient Reward Integrity violation rather than a residual retained reward.

Algorithm 3 enforces two guards before performing a redemption. Let $B_{\min} \in \mathbb{Z}$ be an issuer configured minimum balance threshold, in the same integer minor units, typically 0, below which no redemption is permitted; when $B_{\min} = 0$, no redemption is possible while $B[u] < 0$, providing complete debt enforcement under Assumptions A4 and A6. First, it checks whether the user is within an active grace period hold set by Algorithm 4; this prevents premature redemption in statement cycle systems during the post close window when late refunds may still arrive. Second, it checks that the post redemption balance would not fall below $B_{\min}$, which is typically 0. Together, these guards ensure that both the grace period defense and the negative balance defense are enforced at the redemption boundary. Algorithm 3 evaluates both guards, debits the balance, and records the redemption within a single atomic operation. Combining the check and the debit is necessary rather than incidental, because two concurrent requests that each evaluated the guards against the same pre debit balance could both be authorized and jointly drive $B[u]$ below $B_{\min}$. For deployments that do not invoke Algorithm 4, the grace period mechanism should be explicitly configured based on the reconciliation architecture: if no batch reconciliation component exists under any circumstances, the guard can be made permanently inactive; if any batch reconciliation is performed, even occasionally, the grace hold must remain active.

Two functions support the algorithms. $\text{RuleOf}(t)$, used by Algorithm 1, returns the first earning rule whose predicate t satisfies, and $W_j(\cdot)$ returns the cap window of rule j containing a given accounting period, evaluated at $t.n_s$ on settlement and at the refund's accounting period in Algorithm 2. The pair (j, w) recorded in $\text{Bound}[t.id]$ at settlement is

Algorithm 4: StatementCycleReconcile

Input: User u , billing period n
Output: Adjusted reward set for period n
 $T_n \leftarrow \text{TransactionsInPeriod}(u, n)$;
 $R_{\text{pending}} \leftarrow \text{PendingRefunds}(u, n)$;
 // Phase 1: retroactive clawback for prior period late refunds
 $R_{\text{late}} \leftarrow \text{LateRefunds}(u, n)$;
foreach $(t, x, e) \in R_{\text{late}}$ **do**
 if $t.\text{status} \in \{\text{SETTLED}, \text{PART_REF}\}$ **then**
 $\text{RewardOnRefund}(t, x, e, n)$; // Alg. 2
 // Phase 2: settle on the immutable original amount
foreach $t \in T_n$ **where** $t.\text{status} = \text{PENDING}$ **do**
 $\text{RewardOnSettlement}(t)$; // Alg. 1, on full $t.a$
 // Phase 3: apply same period refunds through the standard path
foreach $(t, x, e) \in R_{\text{pending}}$ **do**
 $\text{RewardOnRefund}(t, x, e, n)$; // Alg. 2
 $\text{RedemptionHoldUntil}[u] \leftarrow \text{StmntCloseDate}(n) + \Delta_{\text{grace}}$

immutable thereafter, exactly as $R_{\text{orig}}[t.id]$ is, and it allows Algorithm 2 to decide whether a later refund falls within the same cap window without reference to the statement cycle.

We define the query functions used by Algorithm 4. Throughout, n denotes the accounting period in which an event is applied, which for a late refund differs from the settlement period $t.n_s$ of the transaction it adjusts. $\text{CurPeriod}(u)$ returns the billing period n such that the current date falls within user u 's active billing cycle $[\text{StmntOpen}(n), \text{StmntClose}(n)]$; a live event handler supplies this value as the accounting period argument to Algorithm 2, whereas Algorithm 4 supplies the period it is reconciling, so that cap accounting is determined by the period being processed rather than by the wall clock date. $\text{TransactionsInPeriod}(u, n)$ returns all transactions whose billing period is n . $\text{PendingRefunds}(u, n)$ returns the set of $(\text{transaction}, \text{refund_amount}, \text{event_id})$ triples whose transaction belongs to period n and whose refund has arrived but has not yet been incorporated into reward computations. $\text{LateRefunds}(u, n)$ returns the set of $(\text{transaction}, \text{refund_amount}, \text{event_id})$ triples where the transaction's billing period is strictly less than n , the refund arrived during period n , and $\text{TotalRefunded}[id] + x \leq t.a$ (i.e., not yet fully processed). By construction, PendingRefunds covers current period transactions and LateRefunds covers prior period transactions, so the two sets are disjoint.

Algorithm 4 addresses the Case II timing gap for statement cycle systems. It runs three phases and mutates no transaction field. It first applies retroactive clawback for prior period transactions that received late refunds during period n , closing the cross cycle gap that is the exploit path in Case II. It then settles every period n transaction still PENDING by invoking Algorithm 1 on the immutable original amount, the status guard and Algorithm 1's idempotency guard together preventing double crediting in hybrid architectures. It finally applies each same period refund through Algorithm 2, so that TotalRefunded , the status, and the balance are updated exactly as for a refund arriving after settlement.

The batch path and the event driven path are thereby equivalent. In both, $R_{\text{orig}}[id]$ denotes the reward computed on the full original amount and is never redefined, every refund is recorded once, and a fully refunded transaction reaches REFUNDED with zero retained reward. An alternative formulation that nets pending refunds by substituting a reduced amount at settlement leaves the cumulative refund counter unset and gives R_{orig} two meanings across the paths, under which a residual positive reward can survive full reversal. The settlement phase

carries no reward eligibility condition, so a transaction whose pending refund equals its full amount is settled and then fully clawed back, recording its terminal state rather than leaving it in PENDING.

We note an important limitation. The grace period protects only against refunds arriving within Δ_{grace} of statement close, whereas merchants with extended or open ended return windows, such as 90 to 365 days or the unlimited windows some warehouse retailers offer, can deliver a refund long after it expires. Algorithm 4's retroactive loop closes that gap within one billing cycle of the refund's arrival, provided per transaction records have not been garbage collected, and it processes all accumulated late refunds at the next active run for dormant accounts. The grace period hold is therefore complementary to per event clawback rather than a substitute for it.

Correctness argument. We present a semi formal proof sketch that the proposed algorithms maintain both invariants. We use the term “semi formal” deliberately: a full machine checked proof would require a complete specification of the issuer's transaction processing pipeline, which is outside our scope. Instead, we state the key properties as lemmas and provide reasoning under stated assumptions.

The argument rests on six conditions. One concerns the environment, namely that the payment network delivers refund events; the remaining five are implementation obligations on the issuer, each independently checkable in a deployed system. **Assumption A1:** Every refund event is eventually delivered to the reward engine (guaranteed by the honest payment network N). **Assumption A2 (Atomic, Idempotent Application):** Each settlement and refund event carries a unique identifier, and the reward engine applies each identified event atomically and idempotently, so that duplicate delivery produces no additional ledger effect. We do not assume exactly once delivery from the network, which commonly provides at least once semantics; the guards in Algorithms 1 and 2 obtain exactly once effect within the engine. Where the network supplies no stable refund identifier, or delivers one merchant refund under several, the ingestion layer must canonicalize events before they reach the engine, since a guard is only as reliable as the identifier it is given. **Assumption A3 (Serialization):** Concurrent refunds on the same transaction are serialized, settlement of a transaction is serialized with refunds on it, and the read and increment of $\text{CapUsed}[u, j, w]$ are atomic at that granularity so concurrent settlements cannot oversubscribe a cap. Concurrent refunds on different transactions are not serialized; the resulting race on CapUsed is benign, as shown below. **Assumption A4 (Account Closure Constraint):** The issuer does not permit user initiated account closure while $B[u] < 0$; outstanding negative balances must be settled before account termination. Issuer initiated closure (fraud, program discontinuation) is outside the scope of our model. **Assumption A5 (Chargeback Atomicity):** Upon receiving the payment network's dispute resolution outcome, the issuer's chargeback pipeline atomically updates the transaction principal record and the associated reward records, triggering the same adjustment logic as a standard refund event. Concretely, the pipeline applies the cumulative target computation of Algorithm 2 to the disputed amount, and where the dispute reverses the transaction in full it records the terminal state as CHARGEBACK in place of the REFUNDED assignment that algorithm makes. The two paths coincide in every other respect, so Lemmas 2 and 4 apply to disputed transactions unchanged. This is what gives the CHARGEBACK state a producer, as required by Eq. (1) and by clause 1 of Refund Reward Consistency. **Assumption A6 (Redemption Gating):** Every channel through which a user can convert reward value does so through Algorithm 3 rather than by debiting the ledger directly. Because that algorithm combines the eligibility check with the debit atomically, a channel routing around it would reopen the extraction window guard (b) is designed to close.

Lemma 1 (Per Transaction Bound). Let used_{pre} denote the value of $\text{CapUsed}[u, j, w]$ for the rule and window to which t is bound, read at the start of Algorithm 1 and $\text{used}_{\text{post}}$ its value on completion. The reward

granted satisfies $R[t.id] \leq \rho_j \cdot t.a$ and $R[t.id] \leq \text{Cap}(u, j) - \text{used}_{\text{pre}}$, both of which follow directly from the min. Separately, the cap counter is never exceeded, since $\text{used}_{\text{post}} = \text{used}_{\text{pre}} + R[t.id] \leq \text{used}_{\text{pre}} + (\text{Cap}(u, j) - \text{used}_{\text{pre}}) = \text{Cap}(u, j)$. The bound on $R[t.id]$ is stated against used_{pre} because Algorithm 1 increments the counter by the amount it grants; stating it against $\text{used}_{\text{post}}$ would be false whenever the cap binds.

Lemma 2 (Cumulative Target Clawback). Let x_t denote the cumulative principal refunded on transaction t . After Algorithm 2 processes any refund, the retained reward satisfies

$$R[t.id] = \left\lfloor \frac{(t.a - x_t) \cdot R_{\text{orig}}[t.id]}{t.a} \right\rfloor \quad (5)$$

The assignment is proportional up to the floor, so the term “proportional clawback” used throughout the paper denotes exactly this rule. Three properties follow. The retained reward is a function of the cumulative refunded principal alone, so it does not depend on how that principal was divided into individual refund events. It never exceeds the proportional entitlement $R_{\text{orig}}[t.id](1 - x_t/t.a)$, since the floor operation only reduces, so no sequence of partial refunds can leave reward above entitlement. And it is zero when $x_t = t.a$, so a fully reversed transaction retains no reward. For a refund arriving within the same cap window as the original settlement, that is when $W_j(n) = w$ for the rule and window to which t is bound, the CapUsed counter is decremented by the clawback, restoring headroom. For refunds arriving after that window has closed, no cap adjustment occurs and clawback operates solely through the balance mechanism.

Lemma 3 (Negative Balance and Grace Period Enforcement). Algorithm 3 declines a redemption under two conditions: (a) if the current date falls within the grace period window $[\text{StmtClose}(n), \text{StmtClose}(n) + \Delta_{\text{grace}}]$ set by Algorithm 4, and (b) if $B[u] - y < B_{\text{min}}$. Guard (a) prevents premature redemption during the configurable grace window; for merchants whose return windows extend beyond Δ_{grace} , this guard provides partial but not complete protection, and Algorithm 4's retroactive LateRefunds loop provides the complementary clawback mechanism for refunds arriving in subsequent periods. Guard (b) creates an automatic debt recovery mechanism: if $B[u] < 0$ after a clawback, no redemption is possible until future reward accruals restore the balance, on the condition stated in Assumption A6 that every redemption channel evaluates the guard.

Lemma 4 (User Level Composition). Assume A1 and A2, so that every delivered event is applied atomically and idempotently and duplicate delivery has no additional ledger effect, that $t.a$ and $R_{\text{orig}}[t.id]$ are immutable after settlement, and that every refund is recorded in $\text{TotalRefunded}[t.id]$ through Algorithm 2. At any quiescent evaluation point, meaning any point at which no delivered event remains unprocessed, the user level net reward depends only on the per transaction records:

$$\begin{aligned} \text{NetReward}(U) &= B[U] + R_{\text{red}}(U) \\ &= \sum_{t \in \mathcal{T}_{\text{settled}}(U)} \left\lfloor \frac{(t.a - x_t) \cdot R_{\text{orig}}[t.id]}{t.a} \right\rfloor \end{aligned} \quad (6)$$

where $x_t = \text{TotalRefunded}[t.id]$. This quantity is invariant to the order in which purchase, redemption, and refund events are interleaved. Sketch: A redemption of y debits $B[u]$ and credits $R_{\text{red}}(U)$ by the same amount, leaving their sum unchanged, so only settlement and clawback affect $\text{NetReward}(U)$. Settling t adds $R_{\text{orig}}[t.id]$ exactly once, since that record is set once and never modified. By Lemma 2, each refund assigns $R[t.id]$ the cumulative target of (5) and debits $B[u]$ by the difference from the previous retained value, so telescoping over any sequence of refunds leaves the transaction's net contribution equal to the target itself, a function of $R_{\text{orig}}[t.id]$ and x_t alone. Summing over $\mathcal{T}_{\text{settled}}(U)$ gives (6), and order independence follows since addition is commutative. Algorithm 3 additionally prevents a user from converting a transient negative balance into extracted value, provided every redemption channel is gated under Assumption A6.

Theorem (Reward Integrity). Let the reward program be transaction separable, meaning that every reward is a deterministic function of a single settled transaction's amount and the earning rule to which it binds, together with that rule's cap over its own accounting window. Let f denote the program's entitlement expressed in the same integer minor units and under the same conservative rounding policy as Algorithm 2, and evaluated under the cap restoration policy of Algorithm 2, namely that headroom is returned when a refund falls in the same cap window as the settlement that consumed it. Under assumptions A1–A6, at any quiescent point after all refund events delivered under A1 have been processed under A2, $\text{NetReward}(U) \leq f(\text{NetSpend}(U), \text{PromoRules})$. During the bounded delivery window guaranteed by A1, the invariant may be temporarily violated; the magnitude of any temporary violation is bounded, in the same integer minor units, by $\sum_{t \in P} R_{\text{orig}}[t.id]$, where P is the set of transactions with delivered but unprocessed refund events. By Lemma 3, redemption during that window is gated by guards (a) and (b) of Algorithm 3, so under A6 a transient excess cannot be converted into extracted value beyond $B_{\min}$, and the excess is removed once the pending events are applied. The bound also requires that per transaction records $R[id]$, $R_{\text{orig}}[id]$, and $\text{TotalRefunded}[id]$ are not garbage collected before the refund arrives; as established in the state machine description, these records must persist for the lifetime of the account. Sketch: By Lemma 1, each transaction contributes at most its rate bounded reward $R_{\text{orig}}[t.id] = \max(0, \min(\lfloor \rho_j \cdot t.a \rfloor, \text{cap} - \text{used}))$, where j is the rule to which t is bound. By Lemma 2, after cumulative refunds of x_t , transaction t retains $R_t = \lfloor (t.a - x_t) R_{\text{orig}}[t.id] / t.a \rfloor$, and by Lemma 4 the user level net reward equals $\sum_t R_t$ independently of event order. We bound the sum one (rule, window) bucket at a time, since f decomposes over the same buckets. Fix a bucket (j, w) with settled transactions B . By Lemma 1, $R_{\text{orig}}[t.id] \leq \rho_j \cdot t.a$, so $(t.a - x_t) R_{\text{orig}}[t.id] / t.a \leq \rho_j (t.a - x_t)$, and monotonicity of the floor gives $R_t \leq \lfloor \rho_j (t.a - x_t) \rfloor$. Since a sum of floors never exceeds the floor of the corresponding sum, $\sum_{t \in B} R_t \leq \lfloor \rho_j \text{NetSpend}_{j,w}(U) \rfloor$ by (2). Separately $R_t \leq R_{\text{orig}}[t.id]$, and Lemma 1 shows $\text{CapUsed}[u, j, w]$, which accumulates the granted R_{orig} values and is decremented only by amounts already removed from the corresponding R_t , never exceeds $\text{Cap}(u, j)$; hence $\sum_{t \in B} R_t \leq \text{Cap}(u, j)$. Taking the smaller bound gives the bucket's term in f , and a fully refunded transaction contributes zero. Summing over buckets yields $\text{NetReward}(U) \leq f(\text{NetSpend}(U), \text{PromoRules})$. Note that when caps bind, the actual net reward may be strictly less than $\sum_t \rho_{j(t)} \cdot (t.a - x_t)$, where $j(t)$ is the rule bound to t , because R_{orig} is capped at settlement time; the algorithms are therefore more conservative than a simple rate times net spend formula would suggest. Because the cumulative target is floored, retained reward can fall below the unrounded proportional entitlement by less than one minor unit per transaction. The discrepancy is therefore always in the direction of under award, and no overpayment tolerance is required in the bound. The five issuer obligations are each falsifiable in a deployed system: idempotent application of identified events (A2), serialization at the stated granularity (A3), closure blocked on a negative balance (A4), a chargeback callback that invokes the refund path (A5), and redemption confined to Algorithm 3 (A6). The separability condition delimits what is proven. Programs whose rewards depend on aggregate behavior rather than on a single transaction, including spend threshold bonuses, sign up and welcome bonuses, tiered accelerators that depend on cumulative period spend, and cross transaction merchant campaigns, are outside the scope of this theorem. Such structures require a reward to be attributed across a set of transactions before proportional clawback is meaningful, and we leave their treatment to future work.

Refund Reward Consistency. For instant event driven systems with a live refund handler, Refund Reward Consistency is bounded by the delivery latency of Assumption A1, and Algorithm 2 then processes the clawback atomically under Assumption A2. For statement cycle systems, same period refunds are applied in the same reconciliation run that settles the transaction, so they are reflected before any reward becomes redeemable, and cross cycle refunds are clawed back retroactively within one billing cycle of arrival. The two architectures therefore offer different guarantees. Live handling provides immediate prevention, since clawback precedes any subsequent redemption.

Table 4

Algorithm to invariant and vulnerability class mapping.

Algorithm	Invariant	Mitigates
1: RewardOnSettle	RI (caps)	Per Txn Bound
2: RewardOnRefund	RI, RRC	V1, V2 ^b
3: AuthorizeAndRedeem	RI, RRC	V3 (partial ^a)
4: StmtReconcile	RI, RRC	V2

^a Guard (a): post-close window; guard (b): negative balance under A4 and A6.

^b For cross cycle refunds, Algorithm 2 mitigates V2 only when invoked by Algorithm 4 or a live handler.

Periodic reconciliation provides bounded eventual restoration, and between the expiry of Δ_{grace} and the next run a redemption request would succeed. An issuer requiring immediate Reward Integrity must therefore handle refunds live or extend the redemption hold through reconciliation.

We note four limitations of the correctness argument as a whole. **Cap accounting.** Headroom is restored only when a refund falls in the same cap window of the same rule as the settlement that consumed it. For later refunds the original window's consumption stands and clawback operates through the balance mechanism, which is safe because the negative balance prevents extraction regardless of cap state. A replacement purchase within the original window may therefore retain headroom released by a partial refund on a transaction later fully refunded after the window closes, which leaves the Reward Integrity bound intact but may slightly over credit within one window. **Concurrency.** Refunds on different transactions bound to the same rule and window race on CapUsed , a race Assumption A3 does not serialize and which is benign for the reason given there. A complete treatment of interleaving under relaxed serialization is left to future work. **Rounding.** Quantities are integer minor units and the retained reward is assigned from cumulative refunded principal, so error cannot accumulate across partial refunds and the target reaches zero exactly on full reversal. The floor in (5) makes any residual an under award bounded by one minor unit per transaction. No clamp is required for correctness, though an implementation may assert $R[id] \geq 0$ as an alerting check. **Composition.** Each transaction carries a single component bound to one rule; multi category purchases and base rates beyond a bonus cap require the decomposition of Section 3.2, and treating such a program as one component awards less than it permits, which is conservative and leaves Lemmas 1 through 4 unchanged. Table 4 maps each algorithm to the invariants it enforces and the vulnerability classes it mitigates.

6.1. Tiered defense architecture

Our defensive algorithms naturally organize into a three tier defense model that mirrors the three tier spectrum of issuer behavior observed in our case studies. Fig. 6 illustrates the two primary invocation paths and the cross cutting redemption gate. In event driven systems, settlement and refund events invoke Algorithms 1 and 2 directly as live handlers, which constitutes Tier 1. In statement cycle systems, Algorithm 4 supplements this arrangement, with a first phase applying retroactive cross cycle clawback through LateRefunds by calling Algorithm 2, a second phase settling transactions still PENDING at statement close by calling Algorithm 1 on the original amount, and a third phase applying same period pending refunds through Algorithm 2, which together constitute Tier 2. Algorithm 3 acts as a cross cutting redemption gate that enforces both the grace period hold (guard a) and negative balance enforcement (guard b).

Tier 1: Prevention (algorithms 1 + 2). Per transaction reward tracking with immutable R_{orig} records and proportional clawback on every refund event eliminates V1 (Refund Insensitive Rewards, Case I) entirely and provides the per transaction linkage that V2 systems lack. This tier requires no timing coordination, no grace periods, and no batch reconciliation; it operates on a per event basis. The robust issuers in

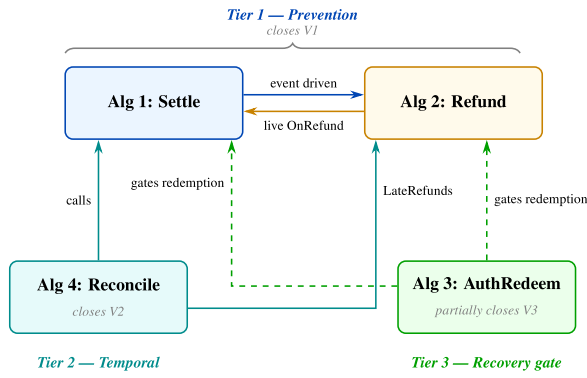


Fig. 6. Algorithm interaction paths across the three defense tiers.

Case III-A (C, D, and E) implement this tier, which is why the DDRA pattern did not succeed against them in our tests, irrespective of when the refund arrived.

Tier 2: Temporal enforcement (Algorithm 4). For statement cycle architectures that batch reward computation, Tier 1 alone is insufficient because a timing window exists between reward computation and refund arrival. Algorithm 4’s three phase reconciliation, comprising retroactive clawback via LateRefunds, settlement of transactions still PENDING, and application of same period refunds through the standard refund path, closes the Case II timing gap. Algorithm 3’s grace period hold (guard a) provides additional protection during the Δ_{grace} window. This tier mitigates V2 (Temporally Scoped Adjustment) for batch architectures.

Tier 3: Recovery (negative balance + A4). Even when Tiers 1 and 2 are imperfect, permitting $B[u]$ to go negative and enforcing Assumption A4 (no account closure while $B[u] < 0$) provides eventual consistency, provided every redemption channel is gated under Assumption A6. The user cannot extract the negative balance debt, and future reward earnings must first repay it. This tier addresses V3 (Inconsistent Negative Balance Handling) and provides a safety net for residual windows that Tiers 1 and 2 do not fully cover, such as the batch cycle window between Δ_{grace} expiry and the next reconciliation run.

Complementary detection. Our prevention focused tiers complement detection based approaches such as PromoGuardian (Li et al., 2025) and behavioral fraud analysis (Chu et al., 2023; Wang and Zhu, 2022), which identify suspicious patterns after the fact. Prevention provides a stronger foundation because: (i) detection systems can have false negatives, especially for honest but opportunistic users whose individual behavior closely resembles legitimate returns; (ii) even perfect detection cannot recover rewards that have already been redeemed and transferred out; and (iii) correct by construction logic reduces operational burden. Detection remains valuable as a fourth layer, particularly for identifying coordinated abuse at scale.

7. Discussion

Reward programs pursue objectives beyond rewarding net spend, including customer acquisition, the formation of card usage habits, cross selling, and sustained engagement. A defense that secured the ledger at the expense of those objectives would be of little use, so we first establish that no such conflict arises. Reward Integrity constrains only the relationship between rewards and net spend; it neither reserves rewards for high spending customers nor withholds them from ordinary cardholders, so a program can pursue acquisition or engagement and still satisfy it, since the property excludes only rewards paid on spend that has been fully reversed. A user who repeatedly purchases, redeems,

and returns within a period works against those same objectives, building no spending habit and creating no relationship into which other products can be cross sold. An issuer may nonetheless accept some reward leakage as an acquisition cost. Our concern is that the leakage in Cases I and II is uncapped by any mechanism we could observe, and we cannot determine from external testing whether issuers intend, monitor, or price this exposure.

7.1. Practical deployment considerations

Implementing the proposed algorithms in existing reward engines raises several practical considerations. First, per transaction reward tracking (the $R[id]$ record) requires that the reward engine maintain a link between each reward credit and its originating transaction identifier. Systems that compute rewards in aggregate (for example, “total grocery spend this month times reward rate”) without per transaction records cannot perform proportional clawback and would need schema changes. The same constraint applies to the Case II design, whose reward computation is cycle scoped and appears to lack per transaction linkage. Closing that gap with Algorithm 4 is therefore not a localized patch but presupposes the per transaction substrate that the robust issuers in Case III-A already maintain, so remediating a V2 engine begins with introducing that substrate. Still within this first requirement, the redemption guard must be applied at every channel through which reward value can leave the ledger, including cash out, statement credit, checkout redemption, and gift card conversion, since an ungated channel would reopen the extraction window even where the clawback logic is correct. This is the condition stated as Assumption A6. Second, negative reward balances require clear customer communication, which we address in Section 7.2. The robust issuers in Case III-A show that at least three production issuers have judged this trade off worth making, which establishes the feasibility of indefinite negative balance enforcement; we do not have visibility into their customer service outcomes and make no claim about them. The partially robust design of Issuer F (Case III-B) suggests that some issuers have chosen to bound negative balance enforcement as a deliberate trade off between fraud prevention and customer experience. We note that when Assumptions A4 and A6 hold (account closure blocked while $B[u] < 0$, and every redemption channel gated by Algorithm 3), the negative balance mechanism (guard b in Algorithm 3) alone is sufficient for eventual correctness: the user cannot escape the debt. The grace period hold (guard a) is therefore a defense in depth measure that reduces the *residual window* during which a batch cycle user could redeem rewards before the next reconciliation run processes a pending clawback. For event driven architectures with live handlers, guard (a) is unnecessary, as Algorithm 2 processes clawbacks immediately. Issuers deploying pure batch cycle systems without live handlers benefit most from guard (a).

7.2. Effect on legitimate returns

Uniform clawback raises the question of whether customers who return purchases for ordinary reasons are disadvantaged, and whether the mechanism can distinguish strategic users from genuine returners. No such separation is performed, and none is required. The algorithms of Section 6 apply the same rule to every refund and make no determination of why a return occurred. Algorithm 2 removes the portion of the reward tied to the returned amount while the principal is refunded, so the customer ends in the same net position as if the purchase had not been made, and by Lemma 4 that transaction’s contribution to net reward falls to the cumulative target of (5), which is zero for a full return. Because no user is classified as abusive, the mechanism produces no false positives arising from intent classification, though it does not eliminate operational error, since a clawback can still be incorrect if a refund is miscategorised, duplicated, or itself later reversed. A discretionary model that revokes rewards once an issuer determines abuse has occurred instead requires a judgment about intent and may misclassify an ordinary customer, which is what generates complaint

volume, support cost, and reputational risk. Behavioral separation of strategic users from ordinary returners is a detection problem situated above the reward logic and is left to future work.

Two residual effects nevertheless warrant issuer attention. A customer who redeems a reward and then returns the purchase will observe a negative reward balance until future earnings offset it. The accounting is correct, but the experience may be perceived as punitive, and an unexplained adjustment can raise complaint volume and, in the limit, prompt account switching. Issuers should therefore pair clawback with notification at the time of adjustment and with statement language tying the adjustment to the returned transaction. The grace period hold in Algorithm 4 likewise delays reward availability for all users of a batch cycle system, whereas the event driven path requires no such hold. Both are deployment considerations rather than defects. Because proportional clawback withdraws only value conditionally earned on a purchase that no longer stands, the residual exposure is one of communication rather than of entitlement.

7.3. Stated policies versus enforced behavior

Reward program terms commonly reserve the issuer's right to reverse, withhold, or recover rewards following a return. These provisions vary in character. Some are discretionary, permitting revocation where the issuer determines that abuse has occurred; others are procedural, specifying automatic deduction upon return and a negative balance where the rewards were already redeemed. The behaviors observed across our six issuers span the same range, which indicates that the integrity properties of Section 4 capture a distinction issuers already articulate in their own terms. Published terms are cited here as illustrative of industry practice and are not linked to any issuer in our anonymized case studies.

The gap our case studies expose is accordingly one of enforcement rather than of policy, since a clause reserving a right to claw back rewards does not guarantee that the engine performs the corresponding ledger update when a refund posts. Case I illustrates this directly. The affected issuer's published terms reserve a right to reduce or remove rewards on returns, refunds, and chargebacks, extending to rewards already redeemed, while comparable refund driven reversal is documented elsewhere in the same program. The vulnerability is therefore not an absence of clawback intent but a failure to execute a reserved adjustment in a system where the capability demonstrably exists and is absent only from the affected flow. Case II rests on different evidence. Its engine computes rewards on a billing cycle scope that cannot reach a return arriving after statement close, which is an integrity gap relative to Section 4 rather than an enforcement gap, since we did not locate published terms for that issuer establishing a comparable commitment. Our contribution is thus complementary to existing terms rather than a restatement of them. We supply the invariants, state machine, and per event algorithms required to make a stated clawback policy hold by construction rather than through manual review. Discretionary enforcement depends on detection, which may fail to identify honest but opportunistic users whose individual returns resemble ordinary behavior, whereas proportional clawback applied to every refund requires no determination of intent.

7.4. Financial sensitivity analysis

Table 5 scales the per user extraction ceiling established in Section 5.1 across a range of assumed adoption fractions and population sizes. Each row represents a hypothetical adoption scenario rather than a projection, and we make no claim that any particular fraction corresponds to observed behavior.

The projection is $p \times |U| \times \$600$ per year, and neither p nor $|U|$ is estimated from our data. For context, publicly available industry data suggests that major digital wallet providers serve tens of millions of active users, and industry reports estimate that loyalty and promotion

Table 5

Per user extraction ceiling scaled by assumed adoption fraction p and active population $|U|$. Figures in millions of USD.

Abuse fraction p	$ U = 100\text{ K}$	$ U = 1\text{M}$	$ U = 10\text{M}$
0.1%	0.06	0.6	6.0
1.0%	0.6	6.0	60.0
5.0%	3.0	30.0	300.0

fraud collectively accounts for one to three billion dollars in annual losses globally (Li et al., 2025). Returns abuse rates in e commerce are estimated at 5 to 10 percent of transactions in some segments (Zhang et al., 2023; Frei et al., 2020). These industry figures are context for the scale of the domain rather than evidence of the exposure we describe.

The security claim and the economic claim rest on different foundations. The security argument does not depend on how many users currently exploit the behavior. For Case I the exploit is deterministic, since any user completing the purchase, redeem, and return sequence retains the reward, so the relevant property is reliable exploitability rather than a measured adoption rate, much as the relevance of an injection flaw does not rest on how many users have exercised it. The parameters p and $|U|$ in Table 5 bound the economic exposure should a given fraction adopt the behavior, and prior work documents that scripted exploitation of refund and concession workflows spreads rapidly once a method becomes known (Sun et al., 2021). Direct measurement of adoption would require issuer internal telemetry that we do not have, and creating accounts at scale to measure it is barred by the identity verification and KYC and AML requirements discussed in Section 8.

7.5. Sources of cross issuer heterogeneity

The six issuers occupy different points on the robustness spectrum. The factors below are plausible explanations rather than findings about the studied issuers' internal systems, since we inspected no architecture and rely on externally observable behavior and public documentation. First, adjustment architecture tends to track platform age, since batch computation at statement close inherits a cycle scope poorly suited to cross cycle refunds, whereas event driven engines support per transaction clawback naturally. Second, per transaction linkage carries a schema cost, and aggregate designs that compute rewards from category totals cannot perform proportional clawback without re-architecture. Third, competitive pressure to offer instantly redeemable rewards pushes issuers toward releasing value at settlement, which, combined with batched adjustment, produces the timing asymmetry of Case III-B. Fourth, organizational placement matters: when reward logic is owned by marketing or growth functions rather than by payment security engineering, refund handling is more likely to be treated as an accounting afterthought than as a state transition with integrity requirements. Finally, product type shapes recoverability, since a debit cash equivalent reward that can be transferred out of the account, as in Case I, is harder to reclaim than a credit card statement credit that can be offset against a negative balance on the same account. These considerations do not defend the vulnerable designs; they explain why robust and vulnerable behaviors coexist and connect to the program objectives set out at the start of this section, since the same acquisition and engagement pressures that motivate instant and generous rewards widen the exposure when they are not paired with refund aware accounting.

7.6. Regulatory context and scope of applicability

The Consumer Financial Protection Bureau in the United States has shown increasing interest in the transparency and fairness of credit card reward programs, including enforcement actions concerning unfair, deceptive, or abusive acts or practices in rewards administration (Consumer Financial Protection Bureau, 2024). Our findings bear on a

related concern. When a reward engine does not enforce basic integrity invariants, the resulting inconsistency affects not only the issuer's financial exposure but also the consistency of treatment across cardholders, since whether a reward survives a return then depends on incidental timing rather than on stated policy. Issuers that implement proportional clawback and persistent negative balances, as in Case III-A, are better placed to demonstrate consistent treatment.

The invariants and algorithms we present are stated over transactions, refunds, and a reward ledger, and are not specific to cashback. Any incentive program that credits value on a transaction and must reconcile that credit when the transaction is reversed faces the same accounting obligation. Whether other reward modalities exhibit the particular failure patterns we document is an empirical question that our sample cannot address.

8. Ethics, disclosure, and methodology

8.1. Experimental methodology

Our empirical observations are based on personal accounts at a small number of issuers that we legitimately hold, low value purchases (under \$50 per transaction) using standard refund processes solely to confirm reward behavior, and public documentation (terms and conditions, FAQ pages) of major reward and loyalty programs. We did not create fake identities or synthetic accounts, did not attempt large scale or automated exploitation, and did not interact with systems outside of published user interfaces.

8.2. Responsible disclosure

All experiments used only accounts we personally and legitimately hold, involved low value transactions consistent with normal consumer use, and were designed to confirm architectural behaviors rather than to extract financial gain.

Case I (Issuer A). We filed a report through the provider's private bug bounty program on October 25, 2025, including a vulnerability description, steps to reproduce the observed behavior, and a recommendation to implement proportional reward clawback on refund events. The provider acknowledged receipt on October 28, 2025, and confirmed the finding as a known vulnerability on January 14, 2026. More than nine months have elapsed since our initial report, exceeding the standard 90 day coordinated disclosure window. We anonymize the provider and omit implementation details per the program's terms.

Case II (Issuer B). We made sustained good faith efforts to locate a disclosure channel. We searched the provider's public website, security pages, support documentation, and legal notices for a vulnerability disclosure policy, security contact, or bug bounty program; none were found. On April 1, 2026, we contacted customer support by telephone and requested referral to a security or compliance team; this produced no usable contact. Later, we sent disclosure emails to the provider's security@ and abuse@ addresses. As of writing, no response has been received through any channel. In the absence of any discoverable disclosure channel after exhausting reasonable direct contact avenues, we proceed with anonymized publication, consistent with responsible disclosure practice for vendors with no accessible vulnerability reporting pathway (Sun et al., 2021; Wang et al., 2011). The finding is a systemic design pattern (statement cycle batching without cross cycle refund reconciliation) described at the architectural level only, with all identifying details anonymized.

Case III-B (Issuer F). Unlike Cases I and II, Issuer F's reward engine does implement a refund adjustment mechanism: a proportional negative reward entry is created at statement close. The vulnerability we document is a timing asymmetry between instant reward availability at settlement and batched clawback at statement close, not a

complete absence of refund handling. Because recovery occurs automatically during normal continued card usage, we characterize this as a design trade off with security implications rather than an unambiguous exploitable vulnerability. Accordingly, we did not pursue formal vulnerability disclosure for this case. We describe the behavior at the architectural level only, anonymize the provider throughout, and present it as a partial robustness pattern to inform secure reward engine design.

8.3. Limitations

Our study has several limitations that we acknowledge transparently. Most significantly, our empirical scope is small: we examine six issuers, constrained by the accounts we personally hold. This scope reflects a constraint intrinsic to the research domain. Opening a credit or debit card account in the United States requires government issued identity documentation, a Social Security Number, a credit inquiry, and issuer approval under applicable KYC and BSA/AML regulations; it is neither legally nor practically feasible for academic researchers to create large numbers of financial accounts for experimental purposes. This constraint is consistent with our adversary model, which characterizes the threat actor as a single consumer using one legitimate account per issuer. The sample is opportunistic and constrained by account creation, KYC, and AML requirements. It supports case study evidence for the vulnerability classes we define and motivates the defensive algorithms, but it does not support claims about prevalence or industry wide practice. We make no population level claims about prevalence, and the figures in Section 7.4 are adoption scenarios rather than measurements. For Case II, our classification rests on three tested transactions on the same account. A refund arriving before statement close reduced the reward eligible base and was reflected in the reward subsequently posted. Two refunds arriving after close produced no adjustment, one in the following period as a full refund and one several billing cycles later as a partial refund. We did not enumerate every possible refund arrival time, but the contrast between adjustment before close and its absence at two different post close delays and magnitudes establishes the temporal scoping that defines the class. We also make no estimate of realized attacker profitability. Whether the accounting gap is worth exploiting depends on merchant return policies, restocking and shipping costs, the friction of repeated returns, issuer monitoring and account action, and the reversibility of each redemption channel, none of which we measured. Our claims concern reward accounting integrity rather than the economics of exploitation. Finally, our observations were not independently replicated, and we did not test systematically across merchant types, refund magnitudes, redemption channels, or timing offsets.

We lack internal visibility into backend systems, risk models, or fraud detection infrastructure at any issuer. We do not build a scanner or automated tool to infer reward logic at scale. The correctness argument is semi formal in the sense set out in Section 6, and it holds under the assumptions and scope stated there. The grace period Δ_{grace} is issuer configurable and would require calibration against refund arrival distributions we do not have. Section 6 sets out its role relative to retroactive reconciliation. Despite these limitations, we believe the combination of real world evidence (including a vendor acknowledged vulnerability), formal modeling, and actionable defensive algorithms provides sufficient value for the security engineering community.

9. Related work

Our work intersects research from web and systems security, operations and marketing science, and industry studies on promotion and returns abuse. To our knowledge, no prior work explicitly models card and wallet reward engines as state machines with refund aware integrity invariants, nor analyzes cashback specific reward abuse patterns through empirical case studies from production financial systems.

Business logic vulnerabilities. Felmetzger et al. proposed automated detection of logic vulnerabilities by inferring expected workflows

(Felmetsger et al., 2010). Pellegrino and Balzarotti demonstrated black box discovery of logic flaws from network traces (Pellegrino and Balzarotti, 2014). Nabi argued for protecting business application logic integrity in e-commerce systems through security focused frameworks (Nabi, 2005, 2011). Yu et al. propose a framework for modeling and analyzing logic vulnerabilities within e-commerce systems during the design phase (Yu et al., 2023). Chen et al. showed that documentation and implementation inconsistencies in payment syndication services produce exploitable behavior (Chen et al., 2019). Ghorbansadeh and Shahriari proposed detecting logic vulnerabilities by comparing design with implementation (Ghorbansadeh and Shahriari, 2020). Sun et al. propose GPTScan, a framework that detects logic vulnerabilities by combining large language models with traditional program analysis (Sun et al., 2024). These works motivate our state machine model, but they do not consider reward specific semantics.

Payment and E-commerce security. Wang et al. showed that integration errors in online payment flows enable attackers to obtain goods without paying, and similar “infinite money” logic flaws have been documented in web application security contexts where store credits or gift cards can be cycled without limits (Wang et al., 2011). Sun et al. analyzed “Concession Abuse as a Service,” documenting industrialized exploitation of customer service concessions (Sun et al., 2021). Van Goethem et al. demonstrated timeless timing attacks that exploit concurrency to leak secrets over remote connections (Goethem et al., 2020); while the time scale differs by orders of magnitude, both our V2 class and timing attacks share the structural property that a security relevant event falls outside the system’s processing window. Behavioral analysis for online payment fraud detection has been studied including spatial temporal pattern exploitation (Chu et al., 2023), fine grained co-occurrence modeling for payment services (Wang and Zhu, 2022), and hidden Markov models for credit card fraud (Srivastava et al., 2008). Du et al. studied app distribution fraud involving promotion investment manipulation (Du et al., 2022). Bitaab et al. proposed large scale detection of fraudulent e-commerce websites, identifying eight distinct types of non-phishing financial fraud (Bitaab et al., 2023); their follow up work uncovered coordinated campaigns of fraudulent shopping websites operating at scale (Bitaab et al., 2025). These detection oriented approaches highlight the gap between identifying fraud after the fact and the prevention by design methodology we propose: our algorithms ensure that reward logic is correct by construction rather than relying on post hoc anomaly detection. Our Case I can be viewed as a cashback specific variant of the concession abuse patterns: the mis modeled artifact is the reward itself, which is not properly tied to the lifecycle of the underlying transaction.

Returns abuse and promotion fraud. Harris documented consumer exploitation of liberal return policies (Harris, 2010). Frei et al. showed that product returns represent a growing problem with significant financial impact (Frei et al., 2020, 2023). Akturk et al. modeled return countermeasures (Akturk et al., 2021). Ketzenberg et al. applied analytics to segment return behavior (Ketzenberg et al., 2020). Liu and Du studied return policy abuse in e-commerce (Liu and Du, 2023). Zhang et al. provided a multichannel returns fraud framework (Zhang et al., 2023). Merlano et al. examined consumer perceptions of anti fraud interventions (Merlano et al., 2024). Von Zahn et al. proposed machine learning nudges to reduce returns (von Zahn et al., 2022). These works treat rewards as part of aggregate economics rather than as a security critical state machine.

Promotion abuse detection. Vieira et al. examine how consumer program loyalty mediates the financial benefits that retailers derive from cashback strategies (Vieira et al., 2022). Leveraging heterogeneous graph neural networks and hierarchical attention mechanisms, Ghosh et al. propose the GoSage framework specifically designed for incentive abuse detection (Ghosh et al., 2023). Utilizing a risk-scoring methodology, Aprisadianti and Dwiyantri develop a novel application specifically designed to detect promotion abuse fraud (Aprisadianti and Dwiyantri, 2023). Li et al. introduced PromoGuardian, a graph neural network for

promotion fraud detection achieving 93 percent precision in production at Meituan (Li et al., 2025). These detection systems complement our prevention focused approach: they identify abusive patterns after the fact, while we propose logic that prevents the abuse from being possible in the first place.

Cashback economics. Taleizadeh et al. showed that cashback strategies interact with refund policies and credit terms (Taleizadeh et al., 2023, 2024). Wu et al. examined cashback tied to online reviews (Wu et al., 2023). These models assume compliant consumers and do not explore adversarial exploitation of reward accounting logic.

Our work is distinguished from adjacent fraud categories by a specific combination of characteristics: the attacker uses a legitimately held account, conducts real purchases through normal merchant channels, makes no false claims about product condition or non receipt, does not collude with other parties, and exploits a logic flaw in the reward engine rather than a technical vulnerability. Account takeover, synthetic identity fraud, and friendly fraud each lack one or more of these properties, and their mitigations (stronger authentication, identity verification, claim investigation) do not address the reward engine logic flaws we study.

10. Conclusion

The vulnerabilities documented in this paper require no sophisticated tools or insider access; they are exploitable by any consumer who understands the basic mechanics of purchasing, redeeming rewards, and returning merchandise. Yet the defenses are equally straightforward: per transaction reward tracking, proportional clawback triggered by every refund event, and negative balance enforcement that persists until recovered. For the purchase, refund, redemption, and subsequent earning workflows we tested, three of the six issuers exhibited behavior consistent with these patterns, which indicates that such reward logic is technically feasible, though our sample cannot establish how widely it is adopted. The key differentiator between a vulnerable system and a robust one is not the sophistication of fraud detection analytics but rather whether the reward engine treats refund events as first class state transitions that must update the reward ledger.

Through controlled experiments on six production issuer accounts, we documented a three tier spectrum of cashback reward handling on refunds, formalized the underlying flaws as violations of Reward Integrity and Refund Reward Consistency within a state machine model, and proposed defensive algorithms with a structured correctness argument. We believe this work makes a case for treating cashback reward engines with the same engineering rigor that the payments industry applies to authorization and settlement logic. As reward programs continue to grow in financial significance, the integrity of the incentive ledger deserves a place alongside the integrity of the payment ledger in security reviews, compliance assessments, and system design.

CRedit authorship contribution statement

S M Zia Ur Rashid: Writing – original draft, Validation, Methodology, Formal analysis, Data curation, Conceptualization. **Suman Rath:** Writing – review & editing, Validation, Supervision, Project administration, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- Akturk, M.S., Ketzenberg, M., Yıldız, B., 2021. Managing consumer returns with technology-enabled countermeasures. *Omega* 102, 102337.
- Aprisadiani, S.N., Dwiyantri, L., 2023. Promotion abuse fraud detection application development using risk scoring. In: 2023 IEEE International Conference on Data and Software Engineering. ICodSE, pp. 208–213.
- Bitaab, M., Cho, H., Oest, A., Lyu, Z., Wang, W., Abraham, J., Wang, R., Bao, T., Shoshitaishvili, Y., Doupe, A., 2023. Beyond phish: Toward detecting fraudulent e-commerce websites at scale. In: Proceedings of the 44th IEEE Symposium on Security and Privacy. pp. 2566–2583.
- Bitaab, M., Karimi, A., Lyu, Z., Oest, A., Kuchhal, D., Saad, M., Ahn, G. J., Wang, R., Bao, T., Shoshitaishvili, Y., Doupe, A., 2025. ScamMagnifier: Piercing the veil of fraudulent shopping website campaigns. In: Proceedings of the Network and Distributed System Security Symposium. NDSS.
- Chen, Y., Zhang, Y., Wang, Z., Xia, L., Bao, C., Wei, T., 2019. Devils in the guidance: Predicting logic vulnerabilities in payment syndication services through automated documentation analysis. In: Proceedings of the 28th USENIX Security Symposium.
- Chu, G., Wang, J., Qi, Q., Sun, H., Tao, S., Yang, H., Liao, J., Han, Z., 2023. Exploiting spatial-temporal behavior patterns for fraud detection in telecom networks. *IEEE Trans. Dependable Secur. Comput.* 20 (6), 4564–4577.
- Consumer Financial Protection Bureau, 2024. Issue spotlight: Credit card rewards. Consumer financial protection bureau. [Online]. Available: <https://www.consumerfinance.gov/data-research/research-reports/issue-spotlight-rewards-credit-card-programs/> (Accessed 5 April 2026).
- Du, S., Zhao, M., Hua, J., Zhang, H., Chen, X., Qian, Z., Zhong, S., 2022. Who moves my app promotion investment? A systematic study about app distribution fraud. *IEEE Trans. Dependable Secur. Comput.* 19 (4).
- Felmetsger, V., Cavedon, L., Kruegel, C., Vigna, G., 2010. Toward automated detection of logic vulnerabilities in web applications. In: Proceedings of the 19th USENIX Security Symposium.
- Frei, R., Jack, L., Brown, S., 2020. Product returns: a growing problem for business, society and environment. *Int. J. Oper. Prod. Manage.* 40 (10), 1613–1621.
- Frei, R., Zhang, D., Bayer, S., Senyo, P., Gerding, E., Wills, G., Beck, A., 2023. The impact of COVID-19 on product returns management in multichannel retail. Available at SSRN 4503184.
- Ghorbansadeh, M., Shahriari, H.R., 2020. Detecting application logic vulnerabilities via finding incompatibility between application design and implementation. *IET Softw.* 14 (4), 377–388.
- Ghosh, S., Anand, R., Bhowmik, T., Chandrashekar, S., 2023. Gosage: Heterogeneous graph neural network using hierarchical attention for collusion fraud detection. In: Proceedings of the 4th ACM International Conference on AI in Finance. pp. 185–192.
- Goethem, T., Van, Peeters, C., Joosen, W., Nikiforakis, N., 2020. Timeless timing attacks: Exploiting concurrency to leak secrets over remote connections. In: Proceedings of the 29th USENIX Security Symposium. pp. 1985–2002.
- Harris, L.C., 2010. Fraudulent consumer returns: exploiting retailers' return policies. *Eur. J. Mark.* 44 (6), 730–747.
- Jiang, H., Yang, L., Jin, C., 2025. Managing sales agents and product returns while guarding against fake orders. SSRN, [Online]. Available: <https://doi.org/10.2139/ssrn.5923462>.
- Ketzenberg, M., Abbey, J.D., Heim, G.R., Kumar, S., 2020. Assessing customer return behaviors through data analytics. *J. Oper. Manage.* 66 (6), 622–645.
- Li, S., Han, X., Zhang, Z., Hua, M., Gao, S., Liang, Z., Guo, Y., Chen, X., Li, D., 2025. PromoGuardian: Detecting promotion abuse fraud with multi-relation fused graph neural networks. *arXiv preprint arXiv:2510.12652*.
- Liu, H., Du, F., 2023. Research on e-commerce platforms' return policies considering consumers abusing return policies. *Sustainability* 15 (18), 13938.
- Merlano, E.F., Frei, R., Zhang, D., Murzacheva, E., Wood, S., 2024. Consumer perspectives on interventions to combat fraudulent product returns in omnichannel fashion retail. *Int. J. Phys. Distrib. Logist. Manage.* 54 (10), 969–1001.
- MITRE Corporation, 2026. CWE-841: Improper enforcement of behavioral workflow. [Online]. Available: <https://cwe.mitre.org/data/definitions/841.html> (Accessed 5 April 2026).
- Nabi, F., 2005. Secure business application logic for e-commerce systems. *Comput. Secur.* 24 (3), 208–217.
- Nabi, F., 2011. Designing a framework method for secure business application logic integrity in e-commerce systems. *Int. J. Netw. Secur.* 12 (1), 29–41.
- OWASP Foundation, 2026. OWASP top 10 for business logic abuse. [Online]. Available: <https://owasp.org/www-project-top-10-for-business-logic-abuse/> (Accessed 5 April 2026).
- Pellegrino, G., Balzarotti, D., 2014. Toward black-box detection of logic flaws in web applications. In: Proceedings of the Network and Distributed System Security Symposium. NDSS.
- Srivastava, A., Kundu, A., Sural, S., Majumdar, A., 2008. Credit card fraud detection using hidden Markov model. *IEEE Trans. Dependable Secur. Comput.* 5 (1), 37–48.
- Sun, Z., Oest, A., Zhang, P., Rubio-Medrano, C.E., Bao, T., Wang, R., Zhao, Z., Shoshitaishvili, Y., A. Doupe, G.-J. Ahn, Zhang, Y., 2021. Having your cake and eating it: An analysis of Concession-Abuse-as-a-Service. In: Proceedings of the 30th USENIX Security Symposium.
- Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y., 2024. Gptscan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. pp. 1–13.
- Taleizadeh, A.A., Varzi, A.M., Amjadian, A., Noori-Daryan, M., Konstantaras, I., 2023. How cash-back strategy affect sale rate under refund and customers' credit. *Oper. Res.* 23 (1), 19.
- Taleizadeh, A.A., Varzi, A.M., Khorshidi, H.A., pricing, M. Noori-Daryan. Retail, 2024. Cashback and refund decisions in a supply chain with e-shop and direct channels. *J. Revenue Pricing Manag.* 23 (2), 140–163.
- Vieira, V.A., Agnihotri, R., de Almeida, M.I.S., Lopes, E.L., 2022. How cashback strategies yield financial benefits for retailers: The mediating role of consumers' program loyalty. *J. Bus. Res.* 141, 200–212.
- Wang, R., Chen, S., Wang, X., Qadeer, S., 2011. How to shop for free online: Security analysis of cashier-as-a-service based web stores. In: Proceedings of the IEEE Symposium on Security and Privacy. pp. 465–480.
- Wang, C., Zhu, H., 2022. Representing fine-grained co-occurrences for behavior-based fraud detection in online payment services. *IEEE Trans. Dependable Secur. Comput.* 19 (1), 301–315.
- Wu, P., Ngai, E.W.T., Wu, Y., 2023. Impact of praise cashback strategy: implications for consumers and e-businesses. *Prod. Oper. Manage.* 32 (9), 2825–2845.
- Yu, W., Liu, L., Wang, X., Bagdasar, O., Panneerselvam, J., 2023. Modeling and analyzing logic vulnerabilities of E-commerce systems at the design phase. *IEEE Trans. Syst. Man, Cybern.: Syst.* 53 (12), 7719–7731.
- von Zahn, M., Bauer, K., Mihale-Wilson, C., Jagow, J., Speicher, M., Hinz, O., 2022. The Smart Green Nudge: Reducing Product Returns Through Enriched Digital Footprints & Causal Machine Learning. No. 363. SAFE Working Paper.
- Zhang, D., Frei, R., Senyo, P., Bayer, S., Gerding, E., Wills, G., Beck, A., 2023. Understanding fraudulent returns and mitigation strategies in multichannel retailing. *J. Retail. Consum. Serv.* 70, 103145.